

## Opis problema

U današnjem sve globaliziranijem svijetu, sposobnost govorenja više jezika vrijedna je vještina za osobni i profesionalni razvoj. Ljudi različitih pozadina žele naučiti nove jezike kako bi poboljšali svoje izgleda za karijeru, iskustva s putovanja ili kulturno razumijevanje.

Tradicionalne škole jezika mogu biti skupe, stroge u smislu rasporeda ili ograničene u ponudi jezika, dok pronalaženje pravog učitelja jezika može biti značajan izazov.

Ovaj projekt nudi rješenje koje je efikasno i pristupačno svima. Cilj je omogućiti spajanje učenika željnima za učenje pojedinog jezika s odgovarajućim učiteljem, preko web aplikacije. Na temelju detaljno definiranih funkcionalnosti, aplikacija će pružati niz mogućnosti za pretraživanje učitelja, pregled njihovih rasporeda, jednostavnu komunikaciju te priliku za obostrano ocjenjivanje, sve na jednom mjestu.

## Ponudeno rješenje

Aplikacija je zamišljena tako da ovisno o vrsti profila: **učenik** ili **učitelj**, korisnik pri registraciji ispuni određene podatke.

Učenik:

- jezik koji želi učiti
- razina znanja
- preferirani stil podučavanja
- cilj učenja

Učitelj:

- jezici koje podučavaju
- godine iskustva
- kvalifikacije
- stil podučavanja
- slika
- kalendar dostupnosti

Na temelju tih podataka, učenik može pretraživati učitelje koji mu odgovaraju te ih filtrirati po nekim značajkama kao što su jezik, kvalifikacije, stil podučavanja, satnica i kalendar dostupnosti. Kada učenik odabere učitelja, šalje mu zahtjev za lekcijom, koju učitelj može, a i ne mora prihvatiti. U slučaju prihvaćanja, otvara im se chat putem kojeg mogu komunicirati.

S druge strane, korisnik koji se nije registrirao ima uvid u listu svih dostupnih učitelja i njihovih rasporeda, ali bilo koja druga funkcionalnost omogućena je samo registriranim korisnicima.

Naravno, **neregistrirani korisnik** se može u bilo kojem trenu registrirati navodeći osnovne podatke potrebne za korištenje aplikacije i vrstu profila. Autorizacija se provodi preko OAuth 2.0 protokola koristeći tokene.

**Registrirani korisnici** se uvijek mogu prijaviti u sustav putem korisničkog imena i lozinke. Ako je prijavljen u sustav, ima mogućnost odjave.

Osim učitelja i učenika, u sustavu postoje i sistemski administratori. **Administrator** je jedini koji održava platformu i ima najviše ovlasti. On može upravljati korisnicima te brisati i uređivati njihove profile. Također upravlja bazom podataka i nadzire rad sustava.

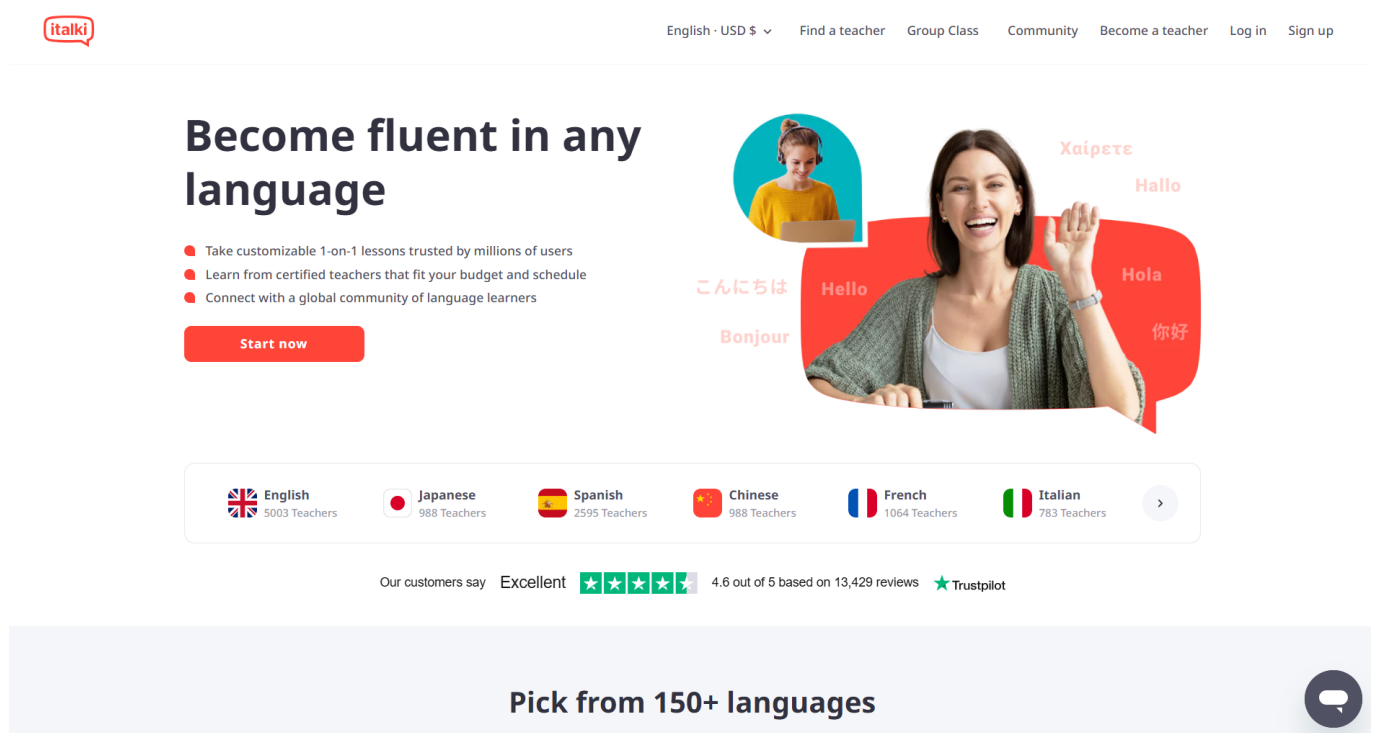
## Korist projekta

Prednost web aplikacije je što može povezati bilo koga, bilo kada i bilo gdje. Aplikacija je namijenjena svima koji imaju želju naučiti novi jezik, ili nadograditi znanje postojećeg. Posebno je korisna za studente, ljubitelje putovanja i stranih kultura, ali i za stručnjake za jezike koji žele proširiti svoje znanje na učenike.

Aplikacija je dizajnirana s mogućnosti prilagodbe funkcionalnosti. Samim time moguće je lako dodavati nove jezike, stilove podučavanja te uređivanje postojećih profila proširivanjem dodatnim značajkama.

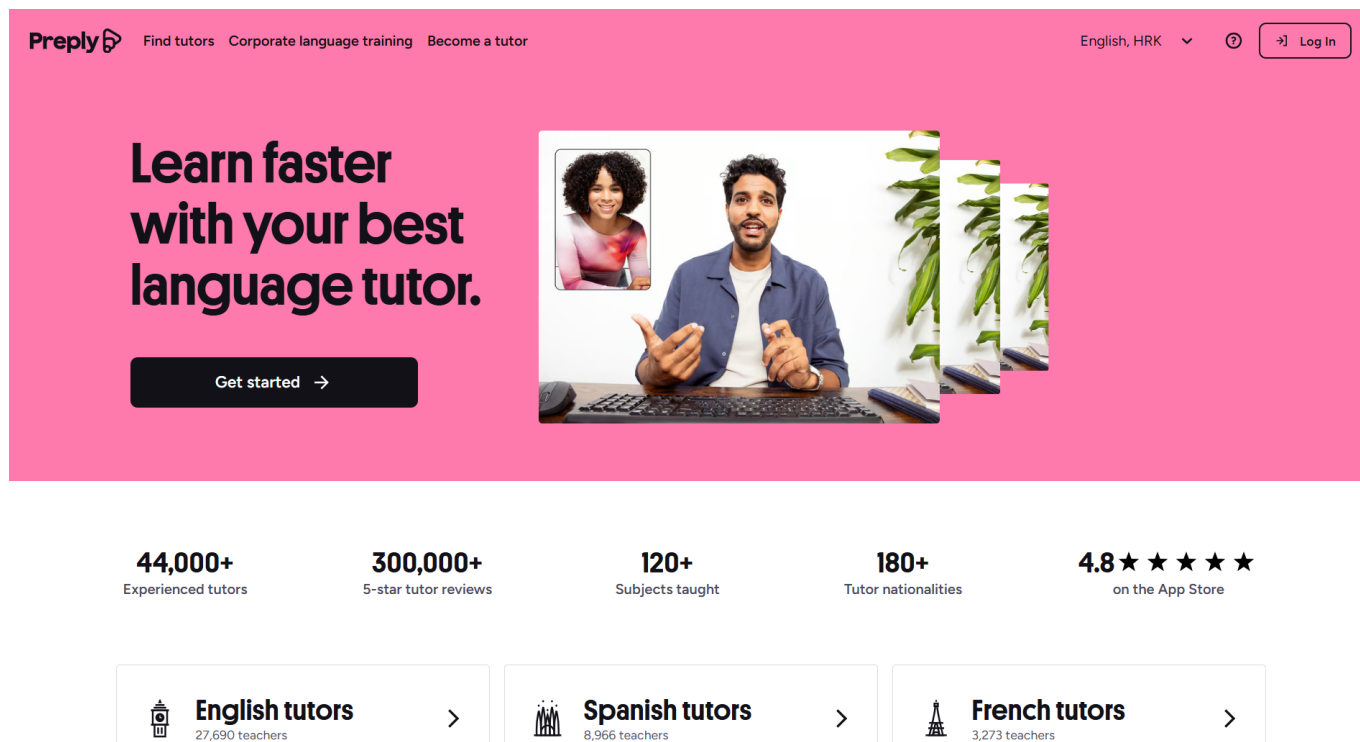
## Postojeća slična rješenja

Na sličnu temu postoje već neka rješenja, a mi smo izdvojili dvije. Primjerice web stranica *iTalki* (<https://www.italki.com/>) korisnicima omogućuje povezivanje s vrhunskim učiteljima, a u bazi ima više od 150 jezika. Izgled njene početne stranice prikazan je na slici dolje.



Slika 1: Izgled početne stranice web stranice Italki

Druga stranica je *Preply* (<https://preply.com/>), a njen izgled prikazan je na slici dolje.



Slika 2: Izgled početne stranice web stranice Preply

Obje stranice su uspješne, ali korisnici naravno odabiru onu koja više odgovara njihovim preferencijama, bila to *Italki*, *Preply* ili neka treća.

## Nadogradnja

Aplikacija ima osnovne funkcionalnosti koje je uvijek moguće nadograditi. Mogla bi se dodati mogućnost videopoziva između učenika i učitelja, uz postojeći chat. Također bi se moglo dodati automatski prijevod stranice na odabrani jezik. Iako je prvenstveno izrađena kao web aplikacija, tu se ostavlja prostor daljnjeg razvoja u mobilnu aplikaciju.

# Funkcionalni zahtjevi

---

Dionici:

1. Registrirani korisnik:

- Učitelj
- Učenik
- Administrator

2. Neregistrirani korisnik

Aktori i njihovi funkcionalni zahtjevi:

1. Učitelj može:

- Registrirati se putem imena, prezimena i e-maila
- Uređivati osobne podatke
- Poslati prijavu u sustav
  - Unosi sve jezike koje priča, godine iskustva, kvalifikacije, način podučavanja, sliku
  - Odabire koliko želi predavati i postavlja svoj kalendar
- Pregledati aktivnosti i povijest
  - Pregled povijesti svih obavljenih lekcija
  - Gleda nadolazeće prijavljene lekcije
  - Filtrirati lekcije po jezicima
- Registrirati novi jezik koji priča

2. Učenik može:

- Registrirati se putem imena, prezimena i e-maila
- Uređivati osobne podatke
- Poslati prijavu u sustav
  - Unose jezik koji priča i želi naučiti, razinu znanja, preferirani način podučavanja
  - Upisuje komentar učitelju zašto želi naučiti
- Pretražuje učitelje
  - Preference učenika
  - Filtrira po jeziku, kvalifikaciji, stil, satnici i kalendaru
- Pregledati aktivnosti i povijest
  - Pregled povijesti svih obavljenih lekcija
  - Gleda nadolazeće prijavljene lekcije
- Nakon lekcije ocijeniti i komentirati učitelja

3. Administrator može:

- Registrirati se putem imena, prezimena i e-maila
- Uređivati osobne podatke
- Upravlja korisnicima
  - Brisati profile

- Uređivati profile

#### 4. Neregistrirani korisnik može:

- Registrirati se putem imena, prezimena i e-maila
- Uređivati osobne podatke

## Nefunkcionalni zahtjevi

---

- Prilagođeno prikazivanje sučelja za različite uređaje (responzivnost).
- Višekorisnički sustav
- Obrazac prilikom registracije/prijave koji provjerava zadovoljavaju li uneseni podaci format i jesu li točni
- Zaštita korisničkih podataka kriptiranjem
- Korištenje https zbog sigurnosti i usklađivanja s normama
- Za implementaciju čavljanja bit će korišteno jedno od postojećih rješenja
- Korištenje gotovih predložaka za ugrađene kalendare - FullCalendar
- Korisnici će biti obaviješteni o stanju zahtjeva kroz sekciju "Lekcije" putem oznaka u boji ili tekstualne poruke
- Vođenje dokumentacije o svim većim promjenama, primjena UML-a

# Obrasci uporabe

---

## Opis obrazaca uporabe

### UC 1 - Registracija korisnika

- Glavni sudionik: Neregistrirani korisnik
- Cilj: Omogućiti korisnicima registraciju u sustav
- Sudionici: Korisnik, Sustav
- Preduvjet: Pristup internetu
- Opis osnovnog tijeka:

1. Korisnik otvara obrazac za registraciju
2. Korisnik unosi ime, prezime, e-mail i vrstu profila
3. Korisnik potvrđuje unos i šalje prijavu
4. Sustav prikazuje poruku o uspješnoj registraciji

- Opis mogućih odstupanja:

1. Ako korisnik ne unese sve potrebne podatke, sustav prikazuje poruku o grešci
2. Ako e-mail već postoji u sustavu javlja se greška o postojećem računu

### UC 2 - Uređivanje osobnih podataka

- Glavni sudionik: Učitelj, Učenik
- Cilj: Omogućiti korisnicima ažuriranje osobnih podataka
- Sudionici: Korisnik, Sustav
- Preduvjet: Korisnik je prijavljen u sustav
- Opis osnovnog tijeka:

1. Korisnik otvara stranicu za uređivanje osobnih podataka
2. Korisnik ažurira željene informacije (ime, prezime, e-mail itd.)
3. Korisnik potvrđuje izmjene
4. Sustav pohranjuje ažurirane podatke

### UC 3 - Osobni podaci za učitelja

- Glavni sudionik: Učitelj
- Cilj: Omogućiti učitelju da pokaže osobne podatke i ako ima potrebu mijenjat
- Sudionici: Učitelj, Sustav
- Preduvjet: Učitelj je uspješno registriran
- Opis osnovnog tijeka:

1. Učitelj otvara obrazac za prijavu (ime i lozinka)
2. Učitelj unosi jezike koje priča, godine iskustva, kvalifikacije, i stil podučavanja
3. Učitelj prilaže sliku i odabire kalendar dostupnosti
4. Sustav sprema podatke i potvrđuje prijavu

- Opis mogućih odstupanja:

1. Ako učitelj ne unese sve potrebne podatke, sustav prikazuje poruku o grešci.
2. Ako slika nije valjana, sustav prikazuje obavijest o grešci i traži novu sliku.

#### UC 4 - Osobni podaci za učenika

- Glavni sudionik: Učenik
- Cilj: Omogućiti učeniku da pokaže osobne podatke i ako ima potrebu mijenjat
- Sudionici: Učenik, Sustav
- Preduvjet: Učenik je uspješno registriran
- Opis osnovnog tijeka:

1. Učenik otvara obrazac za prijavu (ime i lozinka)
2. Učenik unosi jezike koje priča, trenutnu razinu znanja, preferirani stil podučavanja i cilj
3. Sustav sprema podatke i potvrđuje prijavu

- Opis mogućih odstupanja:

1. Ako učenik ne unese sve potrebne podatke, sustav prikazuje poruku o grešci.

#### UC 5 - Pregledati povijest lekcija

- Glavni sudionik: Učenik, Učitelj
- Cilj: Omogućiti korisnicima pregled svih prijašnjih lekcija
- Sudionici: Korisnik, Sustav
- Preduvjet: Korisnik je prijavljen u sustav
- Opis osnovnog tijeka:

1. Korisnik otvara stranicu s poviješću lekcija
2. Korisnik vidi popis svih obavljenih lekcija s datumima i detaljima
3. Korisnik može filtrirati lekcije po jezicima

- Opis mogućih odstupanja:

1. Ako povijest lekcija nije dostupna sustav javlja da prije nije bilo lekcija

#### UC 6 - Registrirati novi jezik

- Glavni sudionik: Učitelj
- Cilj: Omogućiti učitelju da registrira novi jezik koji podučava
- Sudionici: Učitelj, Sustav
- Preduvjet: Učitelj je prijavljen u sustav
- Opis osnovnog tijeka:

1. Učitelj otvara stranicu za registraciju novog jezika
2. Učitelj unosi naziv jezika
3. Sustav sprema uneseni jezik kao dio profila učitelja

- Opis mogućih odstupanja:

1. Ako jezik već postoji u profilu, sustav prikazuje poruku o dupliciranom jeziku

## UC 7 - Ocijeniti i komentirati učitelja nakon lekcije

- Glavni sudionik: Učenik
- Cilj: Omogućiti učeniku da ocijeni i komentira učitelja nakon završene lekcije
- Sudionici: Učenik, Sustav
- Preduvjet: Lekcija je završena
- Opis osnovnog tijeka:

1. Učenik otvara obrazac za ocjenjivanje
2. Učenik unosi ocjenu i komentira iskustvo s učiteljem
3. Sustav pohranjuje ocjenu i komentar

- Opis mogućih odstupanja:

1. Ako ocjena nije unesena, sustav prikazuje poruku da unese ocjenu ili komentar

## UC 8 - Filtriranje učitelja po jezicima, kvalifikacijama, dostupnosti i stilu

- Glavni sudionik: Učenik
- Cilj: Omogućiti učeniku pretraživanje učitelja prema jeziku, kvalifikacijama, dostupnosti i stilu
- Sudionici: Učenik, Sustav
- Preduvjet: Učenik je prijavljen u sustav
- Opis osnovnog tijeka:

1. Učenik otvara pretraživač učitelja
2. Učenik odabire parametre za filtriranje (jezik, kvalifikacije, itd.)
3. Sustav prikazuje popis učitelja koji zadovoljavaju kriterije

- Opis mogućih odstupanja:

1. Ako nema učitelja koji zadovoljavaju kriterije, sustav prikazuje obavijest

## UC 9 - Postavljanje kalendara učitelja

- Glavni sudionik: Učitelj
- Cilj: Omogućiti učitelju da postavi svoj kalendar dostupnosti
- Sudionici: Učitelj, Sustav
- Preduvjet: Učitelj je prijavljen u sustav
- Opis osnovnog tijeka:

1. Učitelj otvara stranicu za postavljanje kalendara
2. Učitelj unosi termine kada je dostupan za podučavanje
3. Sustav sprema unose u kalendar učitelja

- Opis mogućih odstupanja:

1. Ako učitelj nije unio sve potrebne podatke, sustav prikazuje poruku o grešci

## UC 10 - Slanje prijave za lekciju

- Glavni sudionik: Učenik
- Cilj: Omogućiti učeniku prijavu za lekciju s odabranim učiteljem
- Sudionici: Učenik, Sustav
- Preduvjet: Učenik je prijavljen u sustav i odabrao učitelja
- Opis osnovnog tijeka:

1. Učenik otvara profil učitelja
2. Učenik odabire termin iz kalendara učitelja i šalje prijavu
3. Sustav potvrđuje prijavu i obavještava učitelja

- Opis mogućih odstupanja:

1. Ako termin nije dostupan, sustav prikazuje poruku o nedostupnosti termina

## UC 11 - Brisanje korisničkog profila od strane administratora

- Glavni sudionik: Administrator
- Cilj: Omogućiti administratoru brisanje korisničkog profila
- Sudionici: Administrator, Sustav
- Preduvjet: Administrator je prijavljen u sustav
- Opis osnovnog tijeka:

1. Administrator otvara popis korisnika
2. Administrator odabire korisnika i potvrđuje brisanje 3.Sustav trajno briše profil korisnika

- Opis mogućih odstupanja:

1. Ako se korisnik koristi aktivno, sustav prikazuje obavijest o nemogućnosti brisanja

## UC 12 - Pregled nadolazećih lekcija

- Glavni sudionik: Učenik, Učitelj
- Cilj: Omogućiti korisnicima pregled nadolazećih lekcija
- Sudionici: Korisnik, Sustav
- Preduvjet: Korisnik je prijavljen u sustav
- Opis osnovnog tijeka:

1. Korisnik otvara kalendar lekcija
2. Korisnik vidi popis svih nadolazećih lekcija

- Opis mogućih odstupanja:

1. Ako nema nadolazećih lekcija, sustav prikazuje poruku o nepostojanju zakazanih lekcija

## UC 13 - Pretraga učenika po jezicima

- Glavni sudionik: Učitelj
- Cilj: Omogućiti učitelju pretraživanje učenika prema jeziku
- Sudionici: Učitelj, Sustav
- Preduvjet: Učitelj je prijavljen u sustav
- Opis osnovnog tijeka:

1. Učitelj otvara pretragu učenika
2. Učitelj odabire parametre jezika
3. Sustav prikazuje popis učenika koji zadovoljavaju kriterij

- Opis mogućih odstupanja:

1. Ako nema učenika koji zadovoljavaju kriterij, sustav prikazuje obavijest

#### UC 14 - Dodavanje učitelja u omiljene kontakte učenika

- Glavni sudionik: Učenik
- Cilj: Omogućiti učeniku da doda učitelja u omiljene kontakte
- Sudionici: Učenik, Sustav
- Preduvjet: Učenik je prijavljen u sustav
- Opis osnovnog tijeka:

1. Učenik otvara profil učitelja
2. Učenik odabire opciju za dodavanje u omiljene
3. Sustav sprema učitelja u omiljene kontakte učenika

- Opis mogućih odstupanja:

1. Ako je učitelj već u omiljenim kontaktima, sustav prikazuje obavijest o dupliciranju

#### UC 15 - Upravljanje korisničkim računima od strane administratora

- Glavni sudionik: Administrator
- Cilj: Omogućiti administratoru upravljanje korisničkim računima
- Sudionici: Administrator, Sustav
- Preduvjet: Administrator je prijavljen u sustav
- Opis osnovnog tijeka:

1. Administrator otvara popis korisničkih računa
2. Administrator može mijenjati, brisati i deaktivirati račune korisnika

- Opis mogućih odstupanja:

1. Ako korisnik ima aktivne sesije, sustav upozorava administratora

#### UC 16 - Slanje obavijesti korisnicima

- Glavni sudionik: Administrator
- Cilj: Omogućiti administratoru da pošalje obavijest korisnicima
- Sudionici: Administrator, Sustav
- Preduvjet: Administrator je prijavljen u sustav
- Opis osnovnog tijeka:

1. Administrator otvara modul za slanje obavijesti
2. Administrator upisuje obavijest i odabire ciljne korisnike
3. Slanje obavijesti

- Opis mogućih odstupanja:

1. Ako obavijest ne može biti poslana, sustav prikazuje poruku o grešci

## UC 17 - Prikaz povijesti ocjena

- Glavni sudionik: Učenik
- Cilj: Omogućiti korisnicima pregled povijesti ocjena
- Sudionici: Korisnik, Sustav
- Preduvjet: Korisnik je prijavljen u sustav
- Opis osnovnog tijeka:

1. Korisnik otvara stranicu s poviješću ocjena  
2. Korisnik vidi povijest ocjena i komentara

- Opis mogućih odstupanja:

1. Ako nema zapisa ocjena, sustav prikazuje obavijest

## UC 18 - Prijava problema korisničke podrške

- Glavni sudionik: Učenik, Učitelj
- Cilj: Omogućiti korisnicima prijavu problema korisničkoj podršci
- Sudionici: Korisnik, Sustav
- Preduvjet: Korisnik je prijavljen u sustav
- Opis osnovnog tijeka:

1. Korisnik otvara obrazac za prijavu problema  
2. Korisnik unosi opis problema  
3. Sustav sprema prijavu i šalje je podršci

- Opis mogućih odstupanja:

1. Ako je unos nevaljan, sustav prikazuje poruku o grešci

## UC 19 - Komunikacija putem chata

- Glavni sudionik: Učenik, Učitelj
- Cilj: Omogućiti učniku i učitelju komunikaciju kroz chat
- Sudionici: Učenik, Učitelj
- Preduvjet: Učenik i učitelj su povezani kroz zakazanu lekciju
- Opis osnovnog tijeka:

1. Učenik ili učitelj otvara chat vezan uz zakazanu lekciju.  
2. Komuniciraju putem chata o detaljima lekcije (platforma, način plaćanja)  
3. Chat ostaje dostupan dok su oboje prijavljeni u sustav

## UC 20 - Prihvatanje zahtjeva za lekciju

- Glavni sudionik: Učitelj
- Cilj: Omogućiti učitelju da prihvati zahtjev za lekciju

- Sudionici: Učitelj, Učenik, Sustav
- Preduvjet: Učitelj je prijavljen i primio je zahtjev za lekciju
- Opis osnovnog tijeka:

1. Učitelj pregleda zahtjev za lekciju
2. Zahtjev je prihvaćen, sustav obavještava učenika, ažurira kalendar i stvara se chat

## UC 21 - Odbijanje zahtjeva za lekciju

- Glavni sudionik: Učitelj
- Cilj: Omogućiti učitelju da odbije zahtjev za lekciju
- Sudionici: Učitelj, Učenik, Sustav
- Preduvjet: Učitelj je prijavljen i primio je zahtjev za lekciju
- Opis osnovnog tijeka:

1. Učitelj pregleda zahtjev za lekciju
2. Zahtjev je odbijen, sustav obavještava učenika

## UC 22 - Slanje obavijesti učeniku o prihvaćenom zahtjevu

- Glavni sudionik: Učitelj
- Cilj: Obavijestiti učenika o statusu zahtjeva za lekciju
- Sudionici: Učitelj, Učenik, Sustav
- Preduvjet: Učitelj je donio odluku
- Opis osnovnog tijeka:

1. Nakon prihvaćanja zahtjeva, učitelj potvrđuje akciju
2. Sustav šalje obavijest učeniku s odlukom učitelja
3. Učenik prima obavijest o statusu zahtjeva

## UC 23 - Slanje obavijesti učeniku o odbijenom zahtjevu

- Glavni sudionik: Učitelj
- Cilj: Obavijestiti učenika o statusu zahtjeva za lekciju
- Sudionici: Učitelj, Učenik, Sustav
- Preduvjet: Učitelj je donio odluku
- Opis osnovnog tijeka:

1. Nakon odbijanja zahtjeva, učitelj potvrđuje akciju
2. Sustav šalje obavijest učeniku s odlukom učitelja
3. Učenik prima obavijest o statusu zahtjeva

## UC 24 - Pregledavanje svih dostupnih učitelja

- Glavni sudionik: Neregistrirani korisnik
- Cilj: Pregledati sve dostupne učitelje i njihove rasporede
- Sudionici: Sustav
- Opis osnovnog tijeka:

1. Korisnik ima opciju bez registriranja odabrati karticu pogledaj

2. Nakon pritiska na karticu pogledaj dobiva listu svih dostupnih učitelja i njihovih rasporeda

## UC 25 - Odjava iz sustava

- Glavni sudionik: Registrirani korisnik
- Cilj: Odjaviti se
- Sudionici: Sustav
- Preduvjet: Korisnik je prijavljen
- Opis osnovnog tijeka:

1. Korisnik bira Odjavi
2. Nakon pritiska sustav vraća korisnik na početak

## UC 26 - Uređivanje korisničkih profila od strane administratora

- Glavni sudionik: Administrator
- Cilj: Urediti se
- Sudionici: Sustav
- Preduvjet: Administrator je prijavljen
- Opis osnovnog tijeka:

1. Administrator izabire profil
2. Nakon biranja profila ima opciju uredi gdje uređuje profil

## UC 27 - Pregled lekcija

- Glavni sudionik: Učitelj, Učenik
- Cilj: Pregledati lekcije
- Sudionici: Sustav
- Preduvjet: Ima nadolazeće lekcije
- Opis osnovnog tijeka:

1. Korisnik bira Lekcije
2. Nakon odabira ima Nadolazeće lekcije ili arhivu

## UC 28 - Prijava korisnika

- Glavni sudionik: Registrirani korisnik
- Cilj: Omogućiti korisnicima prijavu u sustav
- Sudionici: Korisnik, Sustav
- Preduvjet: Korisnik je registriran
- Opis osnovnog tijeka:

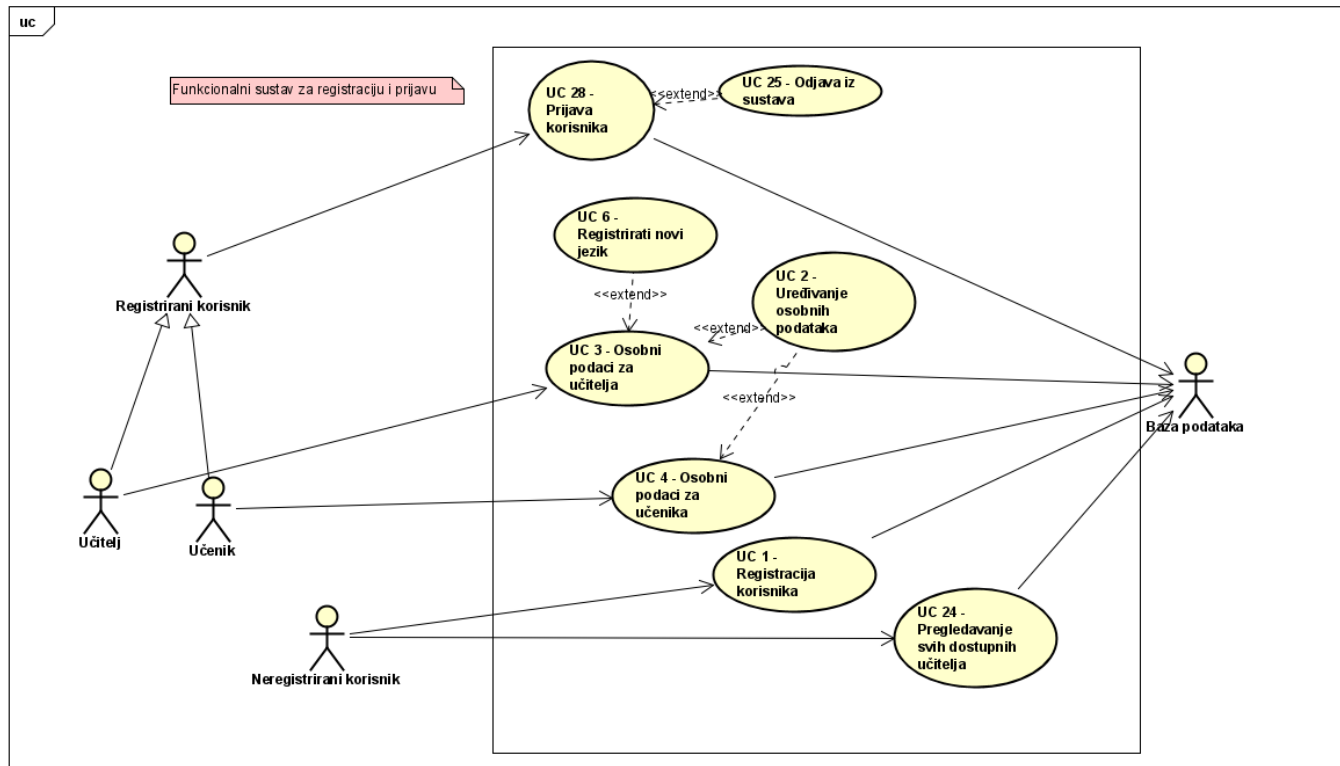
1. Korisnik otvara obrazac za prijavu
2. Korisnik unosi email, lozinku
3. Korisnik potvrđuje unos i šalje prijavu
4. Sustav prikazuje poruku o prijavi

- Opis mogućih odstupanja:

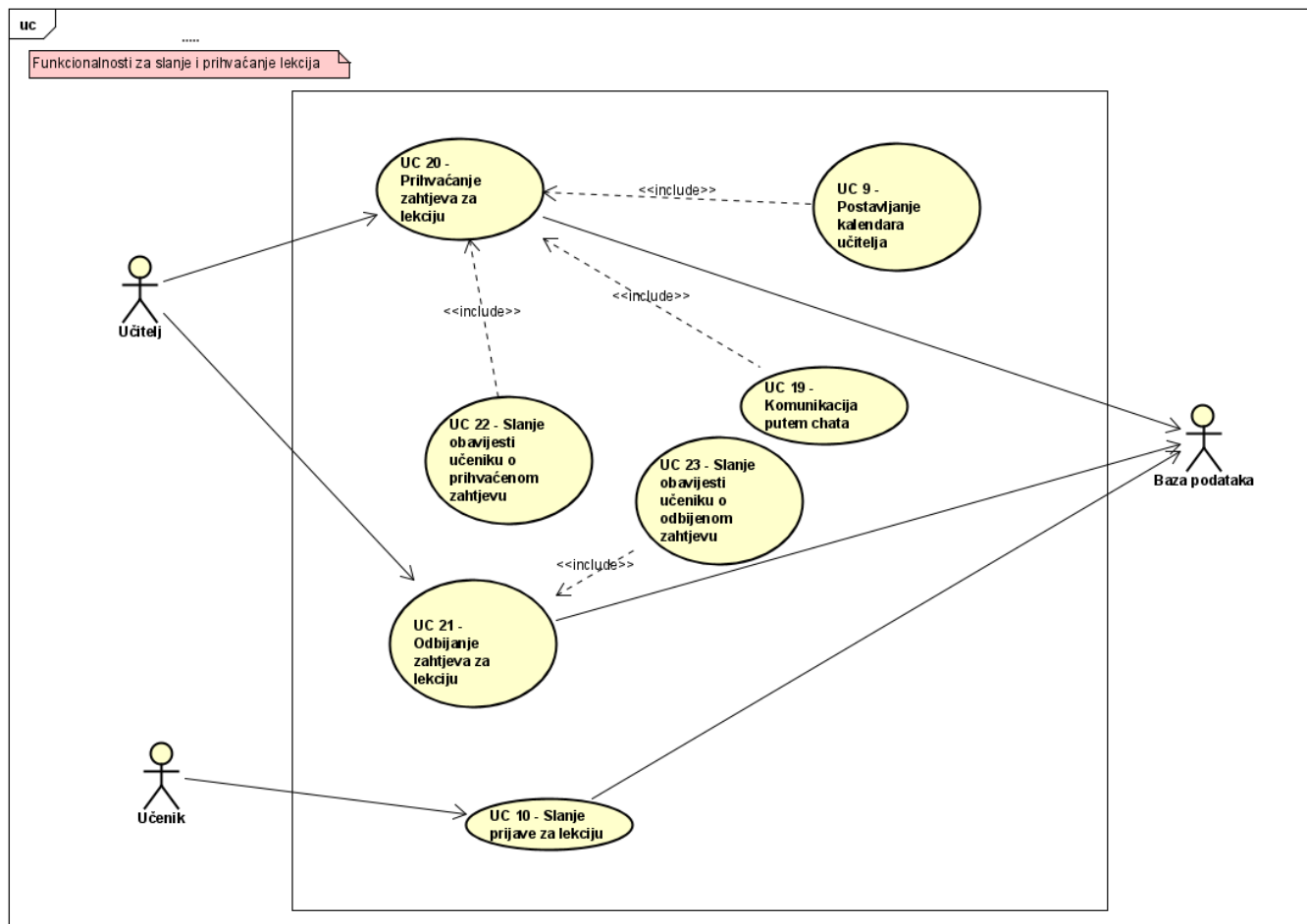
1. Ako korisnik ne unese sve potrebne podatke, sustav prikazuje poruku o grešci

2. Ako e-mail već postoji u sustavu javlja se greška o već postojećem računu

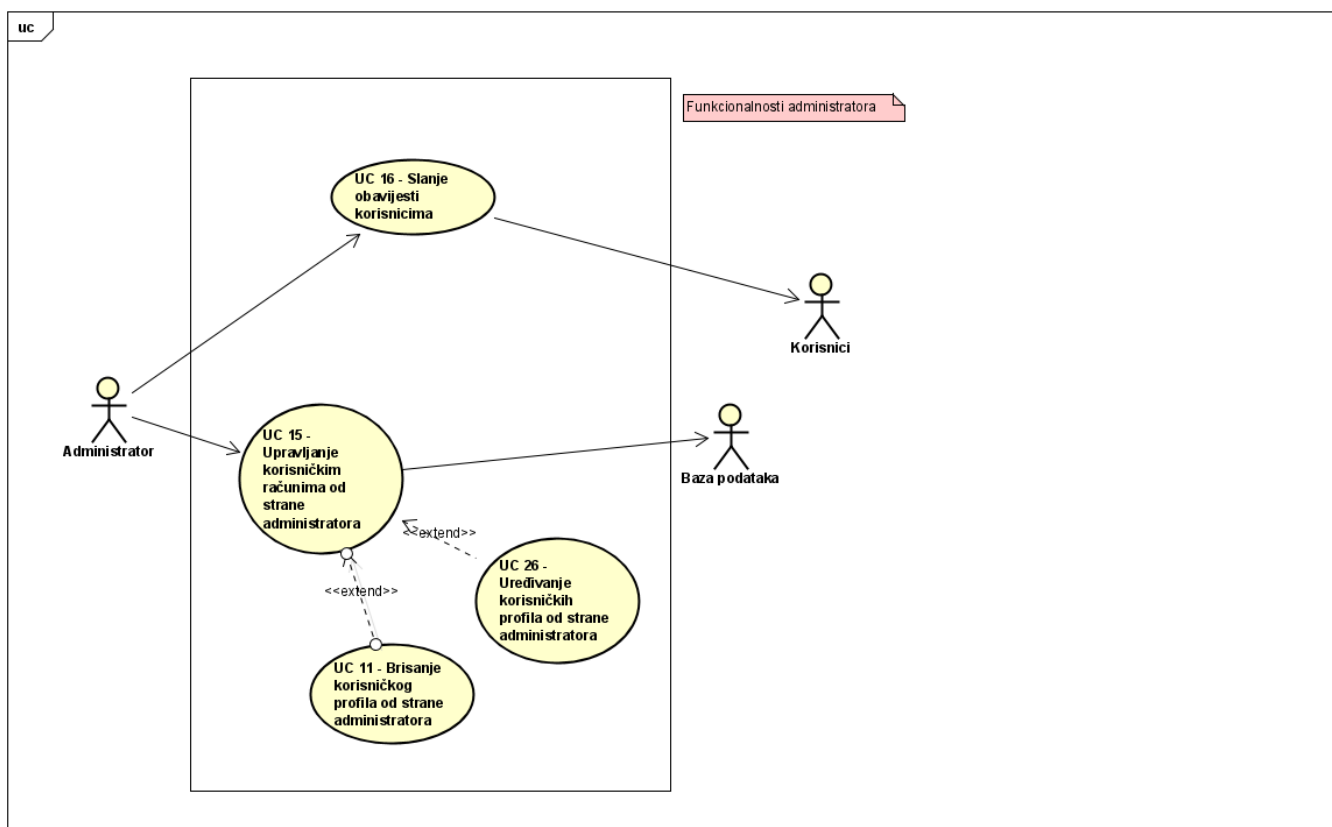
## Dijagrami obrazaca uporabe



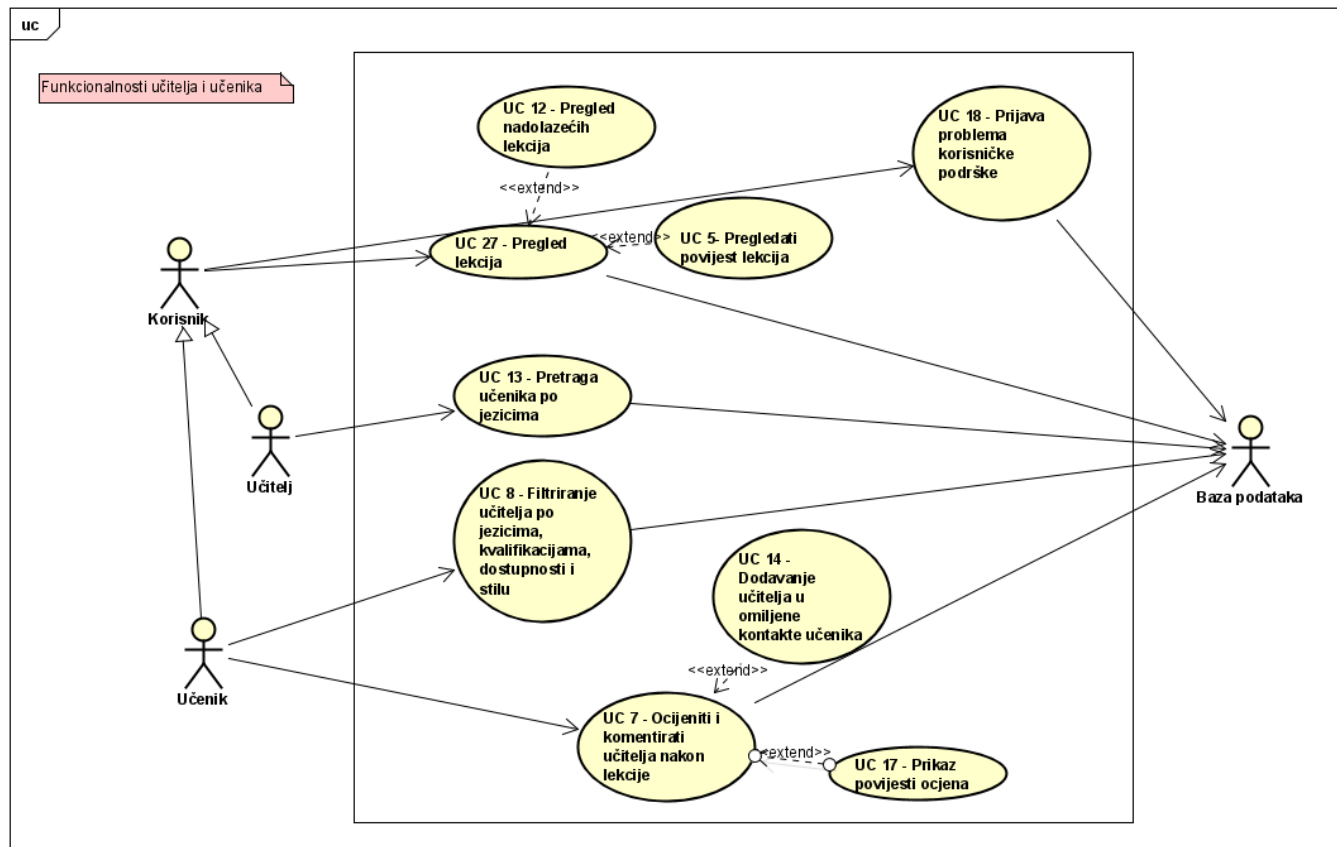
Slika 3: Dijagram prijave i registracije - prikazuje postupak kojim korisnik registrira račun i prijavljuje se u sustav



Slika 4: Dijagram obrasca uporabe- Funkcionalnosti sustava za lekcije.



Slika 5: Dijagram obrasca uporabe- Funkcionalnosti administratora.

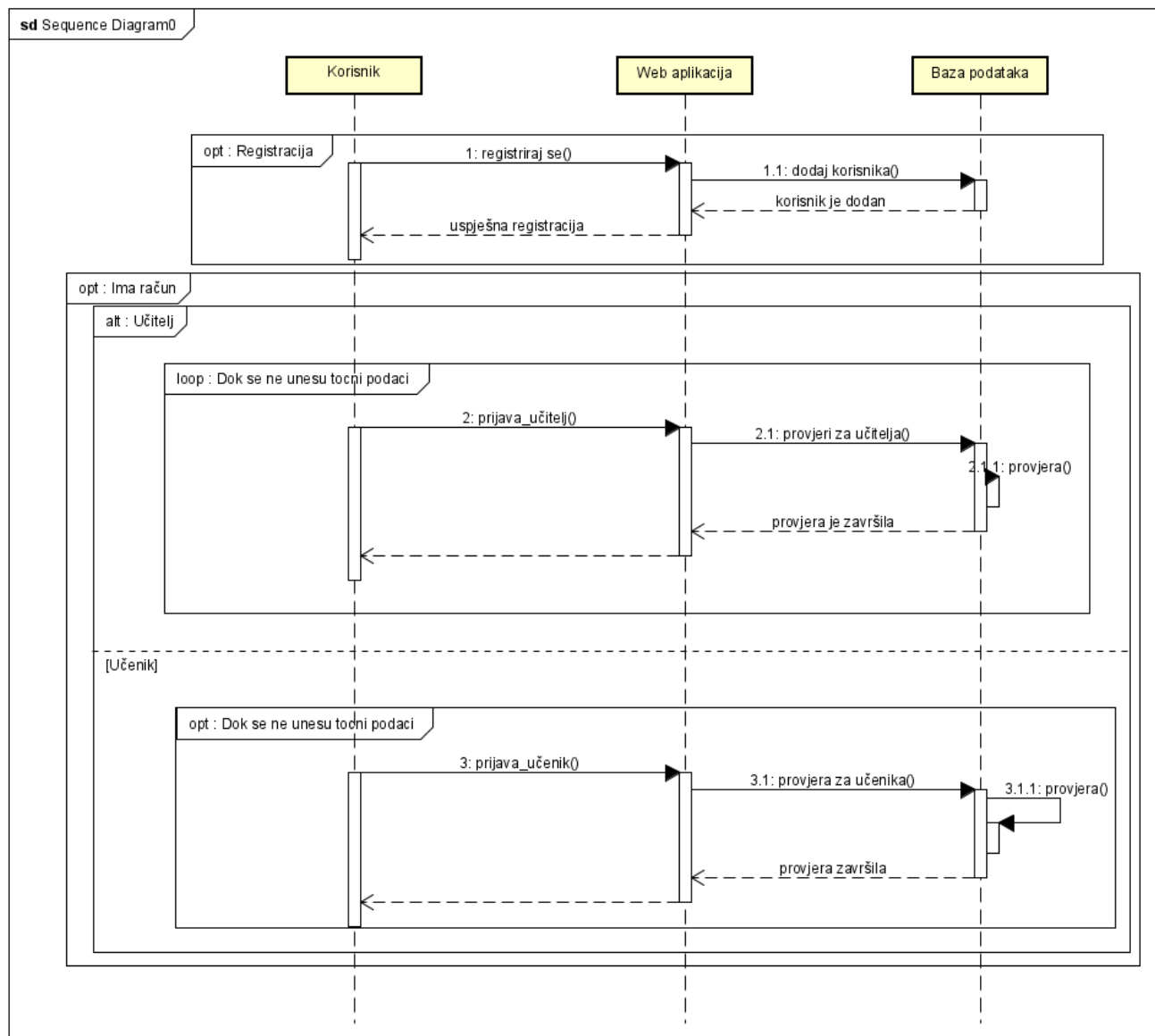


Slika 6: Dijagram obrasca uporabe- Metode učitelja i učenika.

## Sekvencijski dijagrami

### Obrasci uporabe UC1, UC2, UC3, UC4

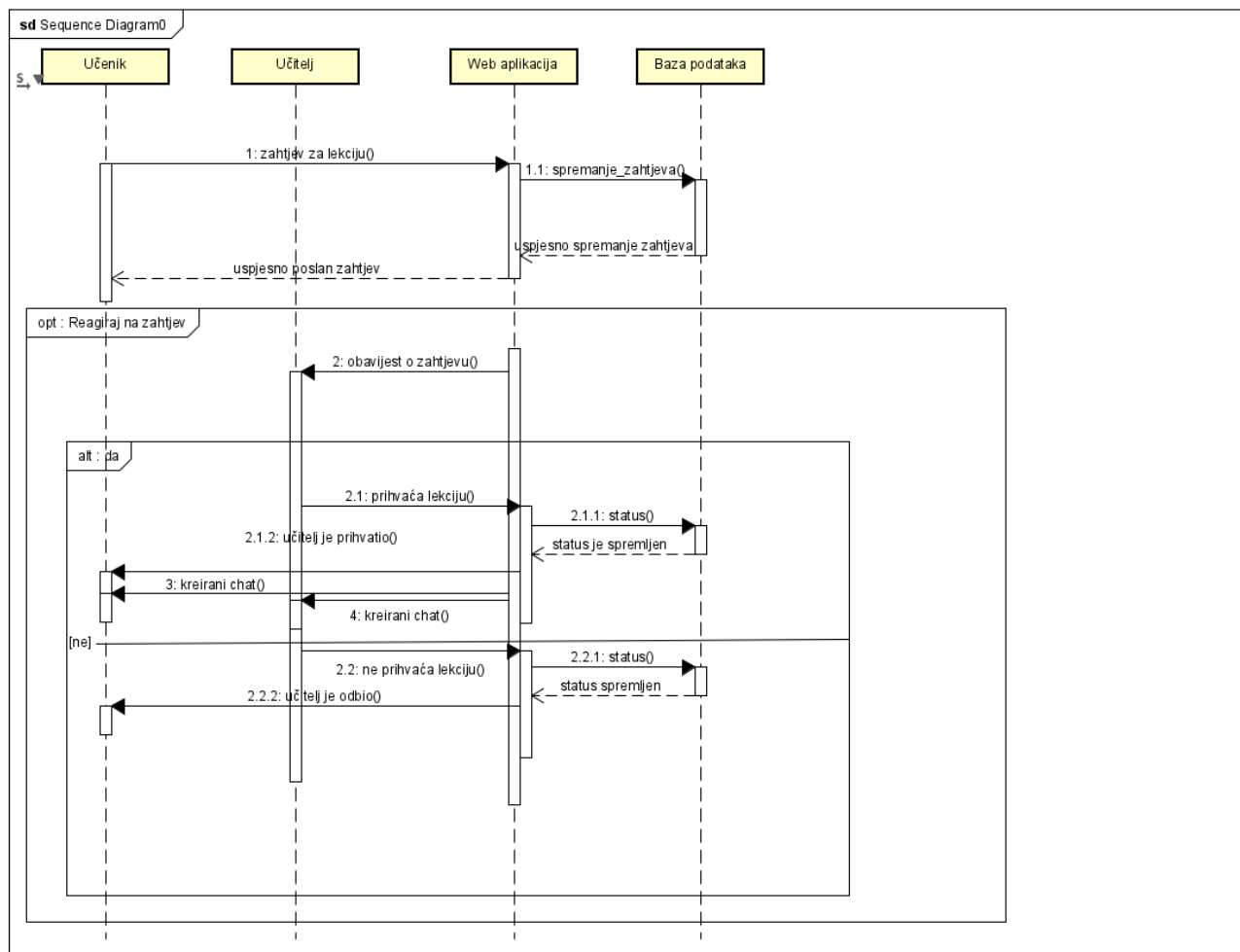
Neregistrirani korisnik šalje zahtjev za registraciju na što sustavom odgovara s formom. Nakon što ispuni formu korisnik šalje sustavu na provjeru, ukoliko je sve dobro stavlja se na bazu podataka. Registrirani korisnik traži prijavu gdje mu sustav vraća odgovor preko obrasca. Nakon što pošalje opet se sve provjerava preko sustav i gleda se baza podatak je li korisnik unio točno, ako je šalje se obavijest da je sve uspješno. On nakon prijave gleda može gledati osobne podatke, lekcije i dr. Imamo učenika i učitelja.



Slika 7: Sekvencijski dijagram-Registracija i prijava

### Obrasci uporabe UC10, UC19, UC20, UC21, UC22, UC23- slanje zahtjeva za lekciju, odgovor učitelja na lekciju i stvaranja chat ako je odgovor prihvaćen

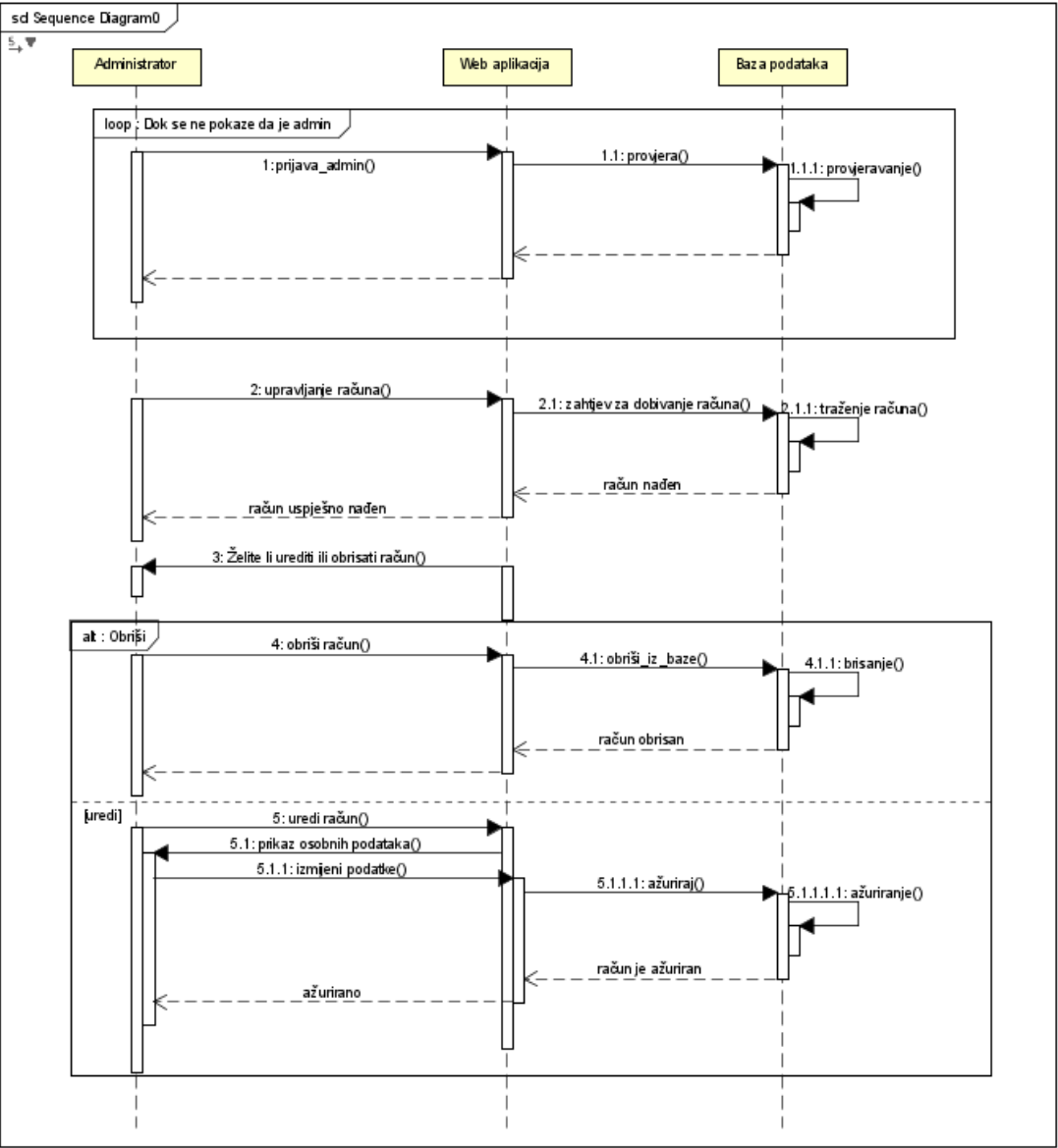
Učenik svaki put kada želi imati neku lekciju mora je poslati sustavu gdje je on bilježi i sprema u bazu. Sustav zahtjev prosljeđuje učitelju od kojeg treba saznati odgovor. Ukoliko učitelj prihvaća zahtjev za lekciju on mora obavijestiti sustav jer kasnije sustav šalje odgovor učeniku. Nakon što učenik dobije odgovor i ako je prihvaćen sustav stvara chat između učitelja i učenika. Druga stvar je ako učitelj odbije zahtjev za lekciju onda se to prosljeđuje sustav koji šalje učeniku da je odbijen i ne stvara se chat između njih.



Slika 8: Sekvencijski dijagram- Funkcionalnost za slanje prijave i odgovor

### Obrasci uporabe UC11, UC26, UC15- upravljanje, uređivanje i brisanje računa od strane admina

Prilikom logiranja administrator mora prvo potvrditi da je administrator zbog privilegija. Nakon prijave administrator može upravljati računima koje zatraži u bazi podataka. Sustav nakon računa ga pita želi li urediti ili obrisati račun. Ako odabere brisanje računa, sustav daje do znanja bazi podataka da se obriše račun i proslijedi poruka kako bi znao da više nema računa. Druga opcija je uredi gdje sustav dostavlja osobne podatke i govori bazu promjenu podataka koje administrator unosi.



Slika 9: Sekvencijski dijagram - Funkcionalnost od računa Administrator

Arhitektura ovog projekta će se temeljiti na korištenju višeslojnog arhitekturnog stila koji će nam omogućavati da si jasnije raspodijelimo odgovornosti među komponentama. Arhitektura nam je olakšala potrebu nad održavanjem, nadogradnjom općenito samog razvoja sustava jer imamo više slojeva od kojih svaki obavlja svoju funkciju. Također nam pomaže tako da članovima tima koji rade na različitim dijelovima bez ikakvog međusobnog utjecaja. Sustav je podijeljen na tri glavna podsustava:

- 1.Web aplikacija
- 2.Poslužiteljski dio
- 3.Baza podataka

**Web preglednik** omogućuje korisniku samu interakciju s web aplikacijom putem HTTPS zahtjeva. On interpretira generirani HTML kôd u oblik koji je razumljiv svim korisnicima.

**Baza podataka** je podsustav u kojem ćemo pohranjivati sve podatke o korisnicima, učenicima, učiteljima, rasporedima, lekcijama i ocjenama. On će komunicirati s back-endom kako bi imali sigurnije i brže pohranjivanje podataka.

**Web poslužitelj** nam obrađuje HTTPS zahtjeve i šalje odgovarajuće podatke ili HTML sadržaj web stranice. On komunicira s klijentom preko web aplikacije. Ona će nam se odvijati preko HTTPS protokola. Također je važan zbog back-enda jer izvršava funkcije, dohvaća podatke kada klijent to želi i obavještava ga o akcijama njegovim.

Aplikacija je podijeljena na:

- **Front-end** je dio aplikacije koji prikazuje korisničko sučelje u web pregledniku. Također nam omogućuje pregled, unos i interakciju s podacima. Napravljen je preko HTML, CSS, JavaScript, odnosno radnog okvira React.
- **Back-end** je dio aplikacija koji obrađuje logiku, obrađuje dobivene zahtjeve i vrši određene akcije, dok komunicira s bazom podataka. Pisan je u programskom jeziku Java, odnosno u okviru Spring Boot.

## Mrežni protokoli

Komunikacija između klijenta i poslužitelja ostvarena je putem HTTPS protokola, gdje klijent šalje zahtjev poslužitelju i on mu na to vraća odgovor u obliku HTML ili JSON podataka. Interna komunikacija između backend sloja i baze podataka ostvaruje se korištenjem PostgreSQL komunikacijskog protokola. Ukratko sustav koristi REST API kako bi komunicirao između front-end i back-end, a za mrežnu se koristi HTTP protokol.

## Globalni upravljački tok

1. Korisnik putem preglednika šalje zahtjev poslužitelju.
2. Poslužitelj prima zahtjev i obrađuje zahtjev.
3. On komunicira sa slojem repozitorija kako bi dohvatio sve potrebne podatke iz baze podataka.
4. Nakon obrade, vraća odgovor klijentu kroz koristeći HTTPS protokol.
5. Klijentski preglednik prikazuje dobivene podatke u obliku razumljivom korisniku.

# Baza podataka

---

U ovom sustavu koristit ćemo relacijsku bazu podataka koja će nam omogućavati jednostavno modeliranje stvarnog svijeta. Baza podataka je implementirana u PostgreSQL zbog jednostavnosti korištenja i zbog toga što je tim već imao iskustva raditi i upoznati se s mogućnostima i ograničenjima sustava. Entiteti stvarnog svijeta se nam prikazani kao zasebne tablice od kojih svaka ima svoje ime i skup drugih različitih atributa. Baza podataka nam pruža da možemo jednostavno pristupiti podacima, umetati, izmjenjivati i pohranjivati.

U našoj bazi podataka koristim sljedeće attribute:

- **Korisnik**
- **Ucitelj**
- **Ucenik**
- **Raspored**
- **Lekcije**
- **Ocjene**

## Opis tablica

**Korisnik** je entitet koji sadrži informacije o prijavi. Sastoji se od atributa: id, name i email. On je povezan preko veze 1:0..1 s tablicom Ucenik preko atributa id\_korisnik. Također je povezan i s tablicom Ucitelj preko veze 1:0..1 s atributom id\_korisnik.

### Korisnik

| Atribut | Tip podatka  | Opis varijable                  |
|---------|--------------|---------------------------------|
| id      | BIGINT(PK)   | Primarni ključ, ID korisnika    |
| name    | VARCHAR(255) | Ime i prezime korisnika         |
| email   | VARCHAR(255) | E-pošta korisnika (jedinствена) |
| picture | VARCHAR(255) | Link do slike profila           |

**Ucitelj** je entitet koji sadrži osobne podatke o učiteljima te veze koje ih spajaju kao korisnicima. Entitet se sastoji od atributa: id, id\_korisnik, jezici, godinelskustva, kvalifikacija, stilPoducavanja, satnica. On je preko veze 0..1:1 i atributa id\_korisnik povezan s korisnikom. Tu je i veza 0..1:N s tablicom raspored preko atributa id\_ucitelj, veza 1:N s tablicom Lekcije preko atributa id\_ucitelj i zadnja veza 1:N s tablicom ocjene preko atributa id\_ucitelj.

### Ucitelj

| Atribut        | Tip podatka   | Opis varijable                                                                                                |
|----------------|---------------|---------------------------------------------------------------------------------------------------------------|
| id             | BIGINT(PK)    | Primarni ključ, ID profila ucitelja                                                                           |
| id_korisnik    | BIGINT(FK)    | ID korisnika                                                                                                  |
| godinelskustva | INT           | Godine iskustva učitelja                                                                                      |
| kvalifikacija  | Kvalifikacija | Kvalifikacije učitelja, ENUM('BACHELORS_DEGREE', 'POST_BACCALAUREATE_DIPLOMA', 'MASTERS_DEGREE', 'DOCTORATE') |

| Atribut         | Tip podatka | Opis varijable                  |
|-----------------|-------------|---------------------------------|
| stilPoducavanja | Stil        | Stil podučavanja učitelja, ENUM |
| satnica         | REAL        | Satnica učitelja                |

**Ucenik** je entitet koji sadrži informacije specifične za učenike. Sastoji se od atributa: id, jezici, trenRazina, stilUcenja, ciljevi, id\_korisnik. Ucenik je povezan s vezom 0..1:1 s tablicom korisnik preko atributa id\_korisnik. Povezan je s vezom 1:N s tablicom Lekcije preko atributa id\_ucenik. Na kraju je povezan vezom 1:N s tablicom Ocjene preko atributa id\_ucenik.

#### Ucenik

| Atribut     | Tip podatka  | Opis varijable                                               |
|-------------|--------------|--------------------------------------------------------------|
| id          | BIGINT(PK)   | Primarni ključ, ID profila ucenika                           |
| id_korisnik | BIGINT(FK)   | ID korisnika                                                 |
| razina      | Razina       | Razina znanja jezika, ENUM('početna', 'srednja', 'napredna') |
| stilUcenja  | Stil         | Preferirani stil podučavanja, ENUM                           |
| ciljevi     | VARCHAR(255) | Ciljevi učenja jezika                                        |

**Lekcija** je entitet koji predstavlja raspored lekcije i općenito sve o lekcijama koje povezuju učenika i učitelja. Sastoji se od atributa: id, timestampPocetka, timestampZavrsetka, status, id\_ucitelj, id\_ucenik. Povezan je s tablicama Ucitelj i Ucenik. S tablicom Ucitelj je povezan s vezom N:1 preko atributa id\_ucitelj, a s tablicom Ucenik je povezan s vezom N:1 preko atributa id\_ucenik. Vremenske intervale za koje učitelj postavlja da je slobodan za podučavanje njihov je status AVAILABLE te kada učenik pošalje zahtjev za lekcijom status se mijenja na PENDING, ako učitelj tad prihvati onda je ACCEPTED, a FINISHED označava da je lekcija održana.

#### Lekcija

| Atribut            | Tip podatka                                                    | Opis varijable                  |
|--------------------|----------------------------------------------------------------|---------------------------------|
| id                 | BIGINT(PK)                                                     | Primarni ključ, ID lekcije      |
| id_ucitelj         | BIGINT(FK)                                                     | ID profila učitelja iz učitelja |
| id_ucenik          | BIGINT(FK)                                                     | ID profila učenika iz učenika   |
| timestampPocetka   | TIMESTAMP                                                      | Vrijeme početka lekcije         |
| timestampZavrsetka | TIMESTAMP                                                      | Vrijeme završetka lekcije       |
| status             | ENUM('AVAILABLE', 'PENDING', 'DENIED', 'ACCEPTED', 'FINISHED') | Status lekcije                  |

**Ocjena** je entitet koji predstavlja ocjene koje učenici daju učiteljima. Sastoji se od atributa: id, id\_ucitelj, id\_ucenik, ocjena, komentar. Ocjene su povezane s tablicom Ucitelj preko veze N:1 i atributa id\_ucitelj. Također entitet je povezan s tablicom Ucenik preko veze N:1 i atributa id\_ucenik

Ocjena

| Atribut    | Tip podatka  | Opis varijable                  |
|------------|--------------|---------------------------------|
| id         | BIGINT(PK)   | Primarni ključ, ID ocjene       |
| id_ucitelj | BIGINT(FK)   | ID profila učitelja iz učitelja |
| id_ucenik  | BIGINT(FK)   | ID profila učenika iz učenika   |
| ocjena     | INT          | Ocjena učenika učitelju         |
| komentar   | VARCHAR(500) | Komentar učenika                |

**Jezik** je entitet koji predstavlja jezike koje se uče na platformi. Sastoji se od atributa: id, name. Jezik je povezana s tablicom UciteljJezici i preko veze 1:N i atributa jezik\_id. Također entitet je povezan s tablicom UcenikJezici preko veze 1:N i atributa id\_jezik.

Jezik

| Atribut | Tip podatka  | Opis varijable                   |
|---------|--------------|----------------------------------|
| id      | BIGINT(PK)   | Primarni ključ, ID tablice Jezik |
| name    | VARCHAR(255) | Ime jezika                       |

**UcenikJezik** je entitet koji predstavlja koje jezike učenik želi naučiti. Sastoji se od atributa: id, id\_jezik, id\_ucenik. UcenikJezik su povezane s tablicom Ucenik preko veze N:1 i atributa id\_ucenik. Također entitet je povezan s tablicom Jezik preko veze N:1 i atributa id\_jezik.

UcenikJezik

| Atribut   | Tip podatka | Opis varijable                       |
|-----------|-------------|--------------------------------------|
| id        | BIGINT(PK)  | Primarni ključ, ID UcenikJezik       |
| id_jezik  | BIGINT(FK)  | ID jezika iz tablice Jezik           |
| id_ucenik | BIGINT(FK)  | ID profila učenika iz tablice Ucenik |

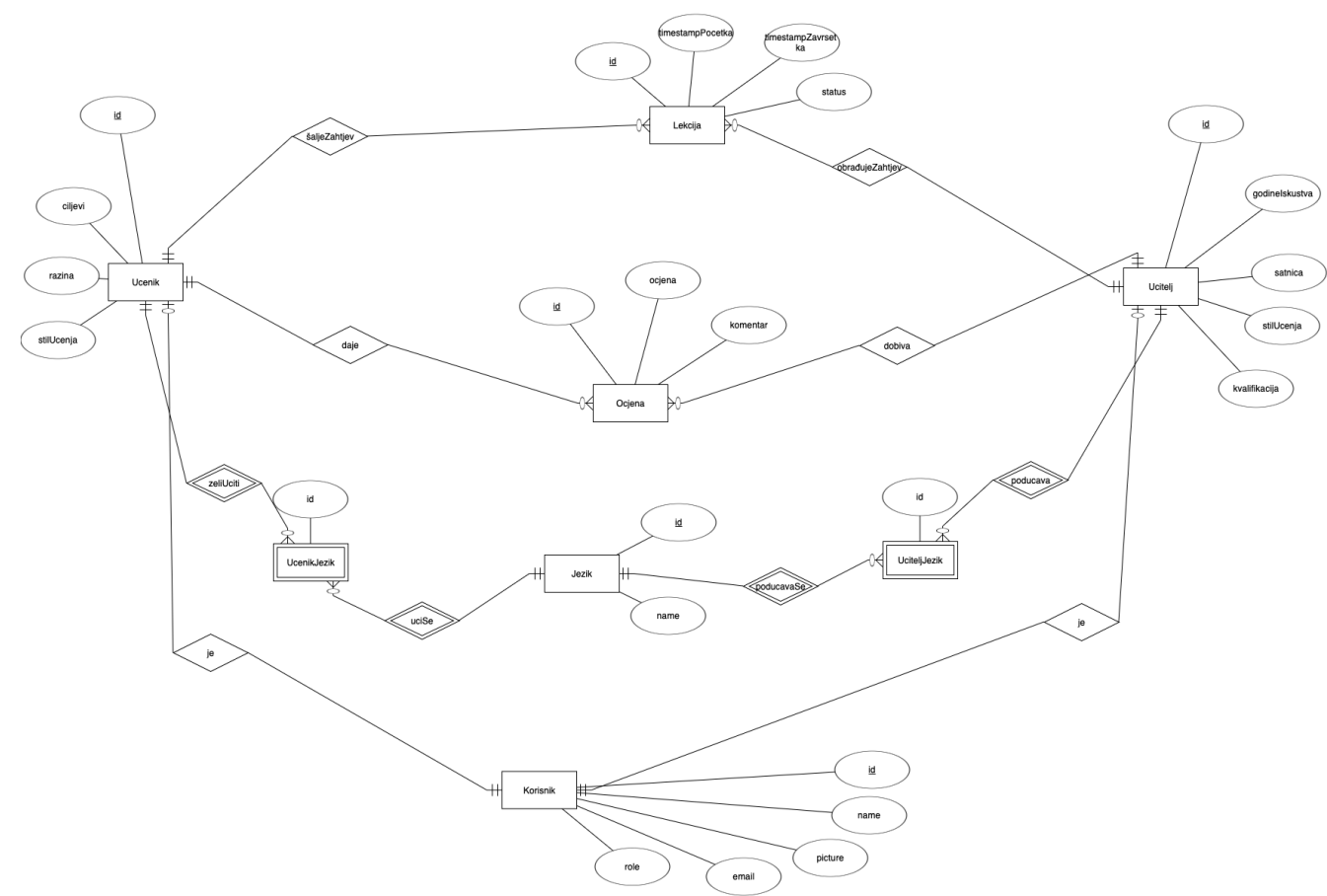
**UciteljJezik** je entitet koji predstavlja koje jezike učitelj podučava. Sastoji se od atributa: id, id\_jezik, id\_ucitelj. UciteljJezik su povezane s tablicom Ucitelj preko veze N:1 i atributa id\_ucitelj. Također entitet je povezan s tablicom Jezik preko veze N:1 i atributa id\_jezik.

UciteljJezik

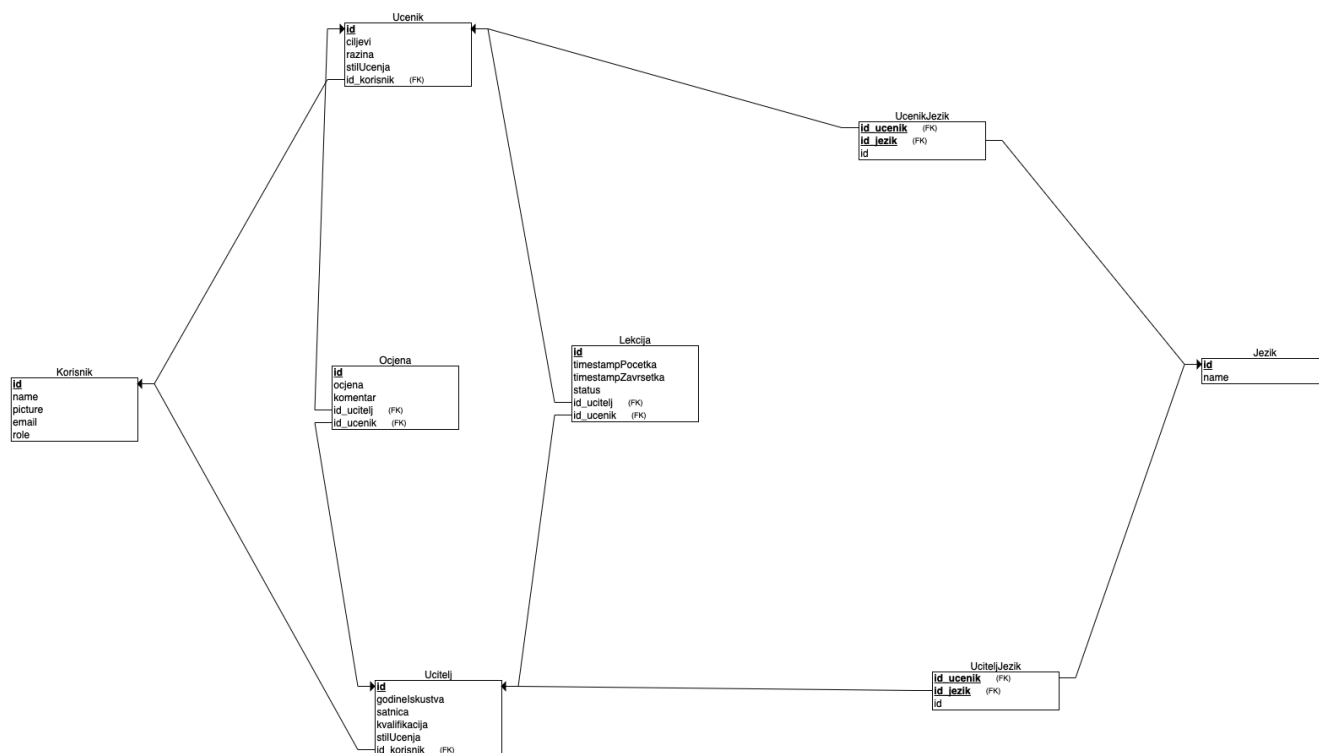
| Atribut | Tip podatka | Opis varijable                  |
|---------|-------------|---------------------------------|
| id      | BIGINT(PK)  | Primarni ključ, ID UciteljJezik |

| Atribut    | Tip podatka | Opis varijable                         |
|------------|-------------|----------------------------------------|
| id_jezik   | BIGINT(FK)  | ID jezika iz tablice Jezik             |
| id_ucitelj | BIGINT(FK)  | ID profila učitelja iz tablice Ucitelj |

Dijagram baze podataka



Slika 10: ER dijagram baze podataka.

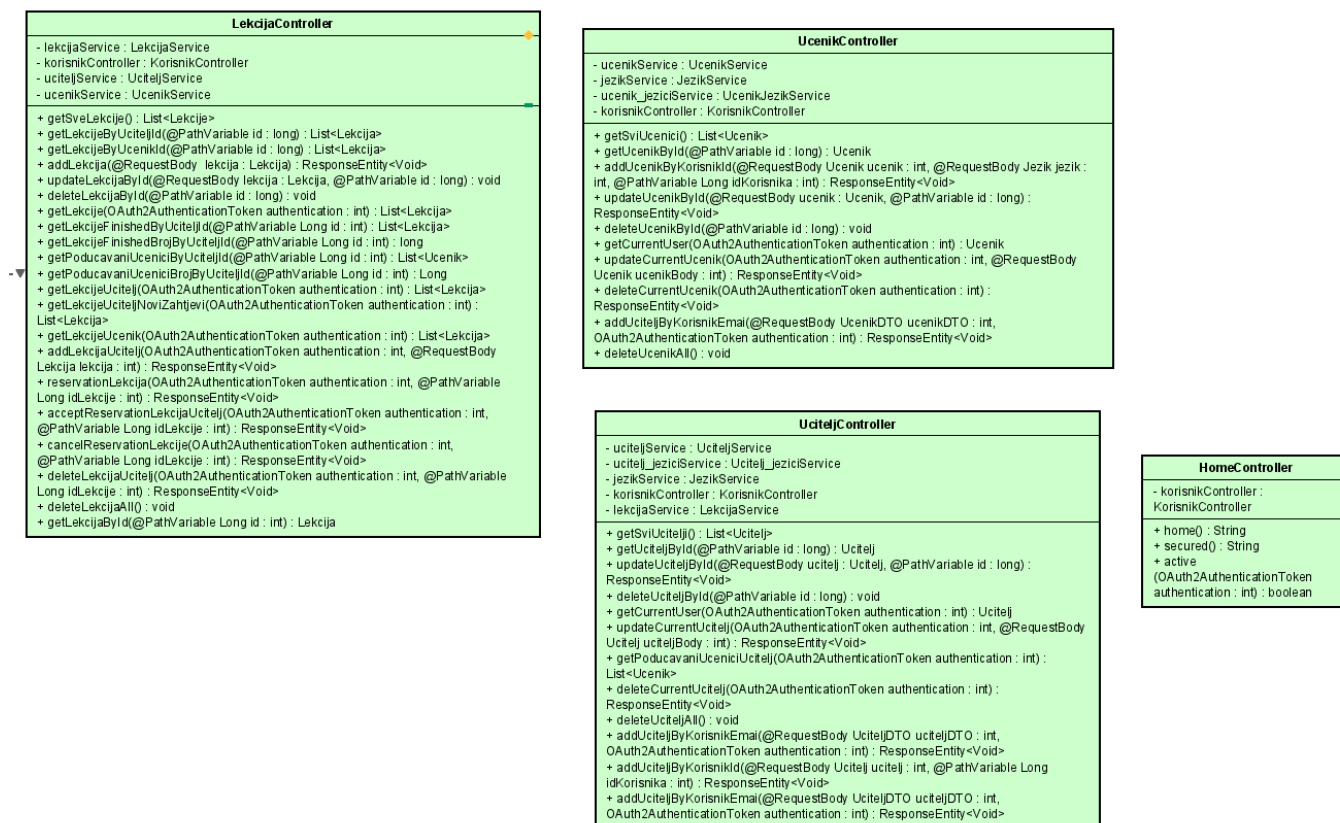


Slika 11: Relacijska shema baze podataka.

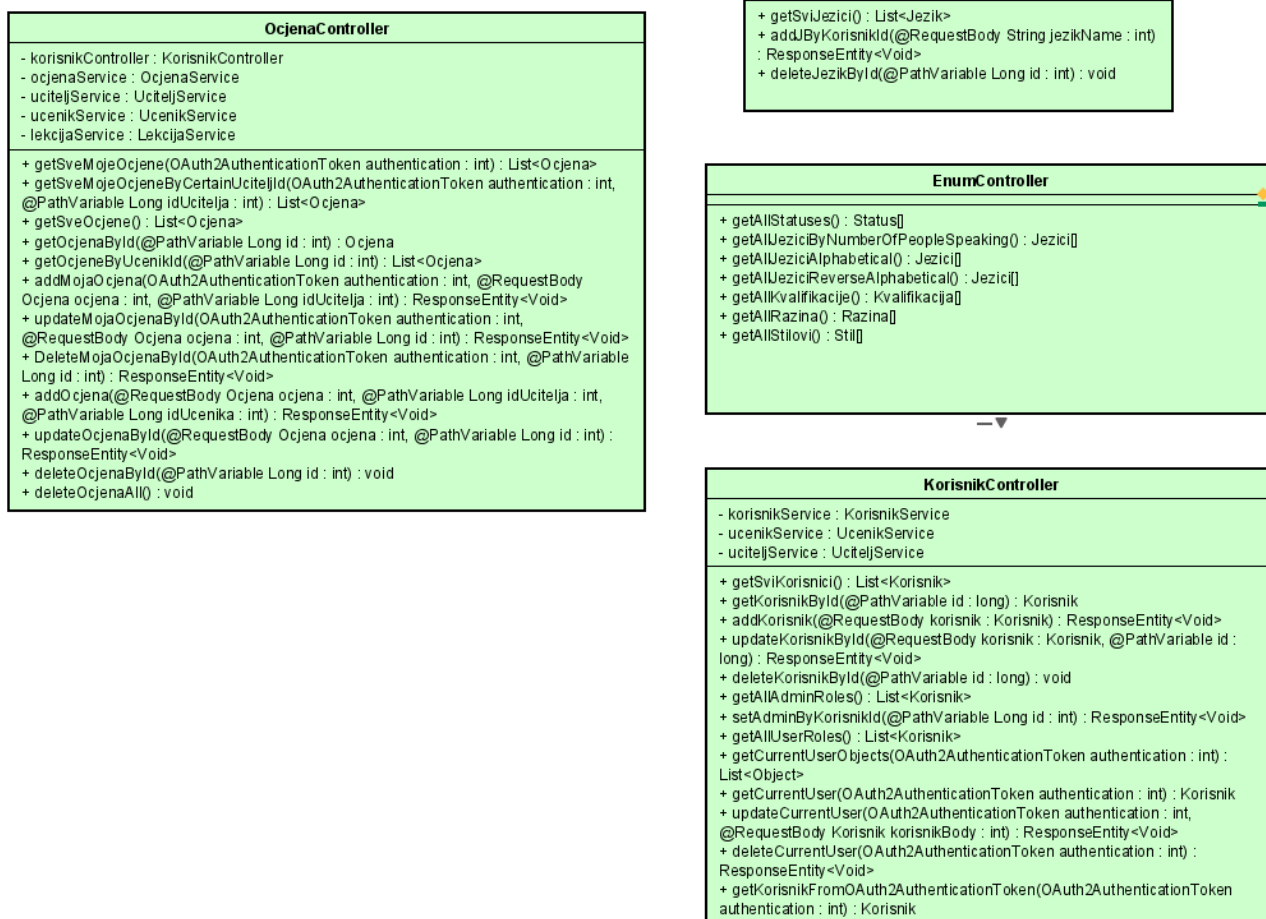
## Dijagram razreda

### Dijagram za Kontrolere

Slika prikazuje sloj kontrolera koji su odgovorni za obradu zahtjeva i komunikaciju s klijentima. Svaki kontroler ima odgovarajući Service koji koristi za izvršavanje poslovne logike. HomeController služi kao početni kontroler s osnovnim metodama za prikaz početne stranice i samog aktivnog korisnika. Također imamo i druge kontrolere sa svojim određenim metodama poput EnumController, JeziciController, KorisnikController, LekcijaController, OcjenaController, UcenikController, UciteljController. EnumController nam služi za dohvaćanje statusa, razina, jezika, kvalifikacija odnosno svih enumeracija. UcenikController i UciteljController su slični controlleri po metodama. Oni imaju metode za dohvaćanje trenutnog korisnika, ažuriranje, brisanje i dodavanje učitelja i učenika, ovisno o kontroleru. Osim toga imaju i metode za dodavanje po određenoj stavci, ažuriranje, brisanje, dodavanje po Id i brisanje. OcjenaController koristi metode vezane za ocjene, odnosno ažuriranje, davanje, brisanje i prikaz liste ocjena. LekcijaController je koristan za pregledavanje završenih lekcija, novih zahtjeva, budućih lekcija, odnosno općenito sve za lekcije. KorisnikController nam je zaslužan za dodavanje, brisanje, prikaz liste i ažuriranje korisnika bilo to učenika, učitelja, administratora i dr. Na kraju imamo i JeziciController za dodavanje novog jezika, brisanje i prikaz.



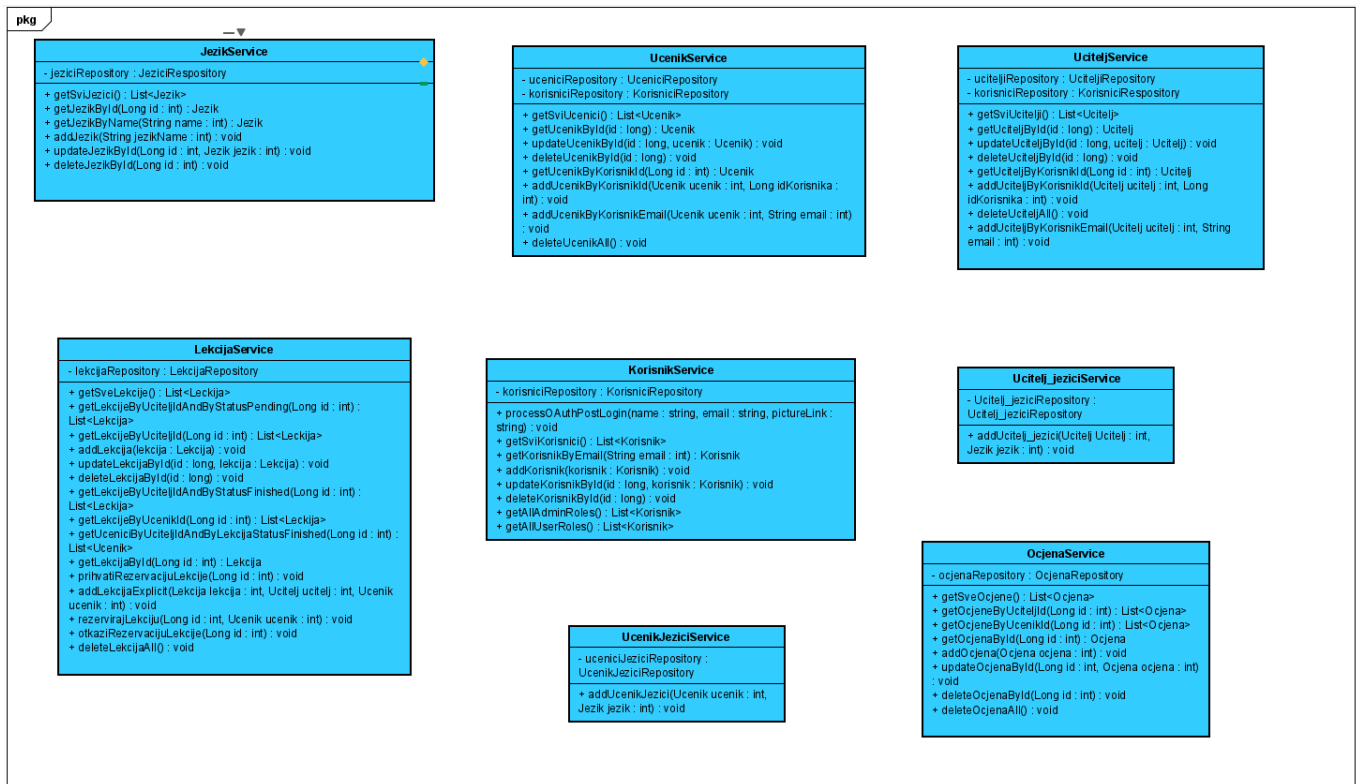
Slika 12: Dijagram kontrolera (1.dio)



Slika 13: Dijagram kontrolera (2.dio)

### Dijagram za Service

Slika prikazuje sloj Service koji sadrži poslovnu logiku aplikacije. Svaki entitet ima svoj servis koji služi kao posrednik između kontrolera i repozitorija. JezikService, KorisnikService, UcenikService, UciteljService, LekcijaService, OcjenaService, UcenikJeziciService i Ucitelj\_jeziciService implementiraju nam metode važne za manipulacija entitetima. Svaki Service koristi repozitorij za pristup bazi podataka. KorisnikService ima dodatnu metodu processOAuthPostLogin koja omogućava obradu podataka tijekom OAuth autentifikacije. Također svaki Service ima metode za brisanje, dohvat, mijenjanje i dodavanje objekata po ID-u.

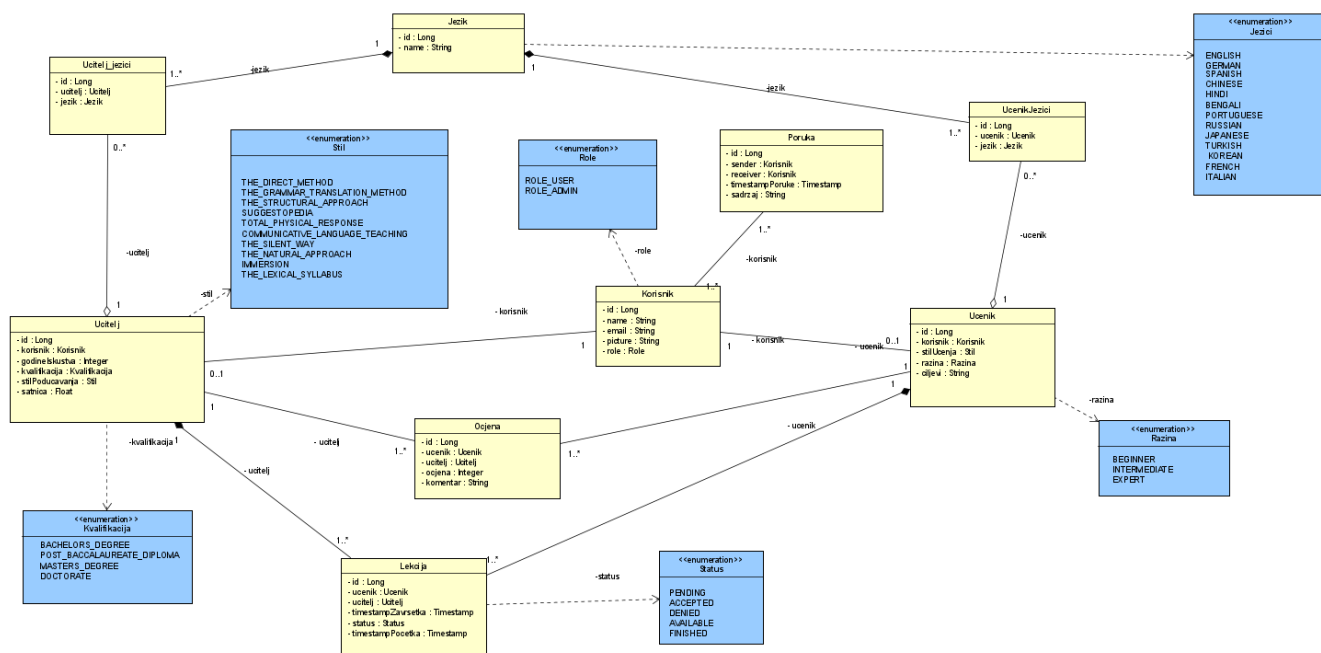


Slika 14: Dijagram servisa.

### Domain dio

Dijagram razreda prikazuje strukturu baze podataka za sustav upravljanja učenicima, učiteljima, lekcijama, ocjenama i rasporedima. Ovaj dijagram uključuje glavne entitete sustava i njihove međusobne odnose, kao i potrebne attribute. Svi razredi predstavljeni na dijagramu direktno preslikavaju entitete baze podataka i koriste se za interakciju s bazom. Razred **Korisnik** predstavlja osnovne informacije o svakom korisniku sustava, uključujući attribute kao što su id, name, email i picture. Svaki korisnik može biti povezan s Učenicom ili Učiteljem. **Učenik** je razred koji opisuje specifične karakteristike učenika, kao što su ciljevi učenja i preko enumeracije Razina možemo vidjeti koliko dobro poznaje jezik. Također imaju povezane jezike koje uče putem veze s enumeracijom Jezici. **Učitelj** je razred koji prikazuje specifične informacije o učiteljima, poput jezika koje predaju (enumeracija Jezici), godina iskustva, kvalifikacija (enumeracija Kvalifikacija) i satnice. **Lekcija** predstavlja zakazan termin između učitelja i učenika. Svaka lekcija sadrži attribute timestampLekcije za označavanje vremena održavanja, status za praćenje statusa lekcije (enumeracija Status), te poveznicu na učitelja i učenika koji sudjeluju. **Ocjena** služi za evidentiranje povratnih informacija učenika o učiteljima. Sadrži attribute ocjena i komentar. Svaka ocjena je povezana s određenim učiteljem i učenikom. **Raspored** bilježi

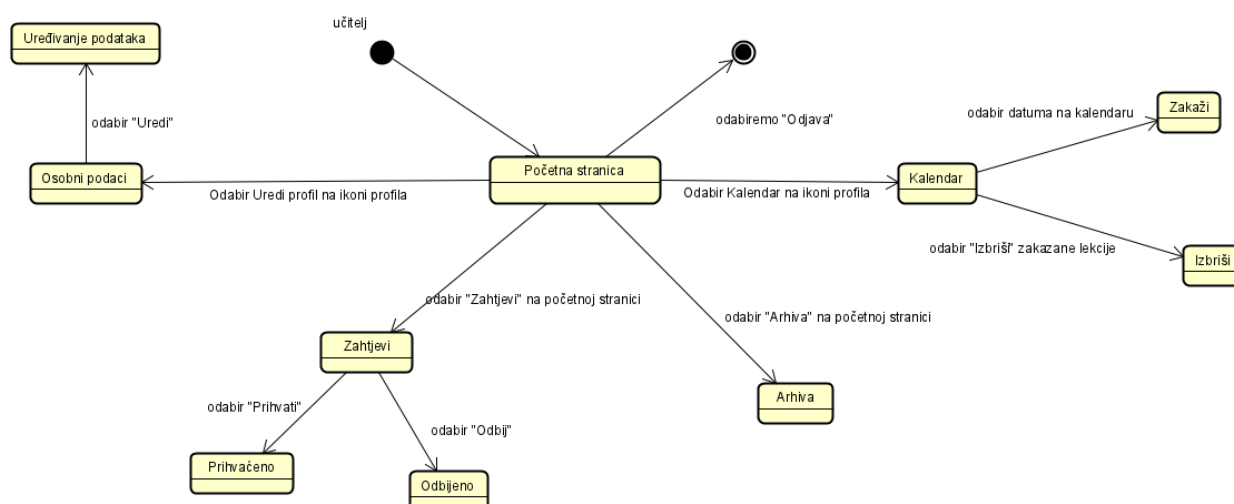
informacije o terminima predavanja koje učitelji održavaju. Sadrži atribute timestampPocetka i timestampZavrsetka koji označavaju početak i kraj termina u rasporedu.



*Slika 14: Dijagram modela.*

## Dijagram stanja

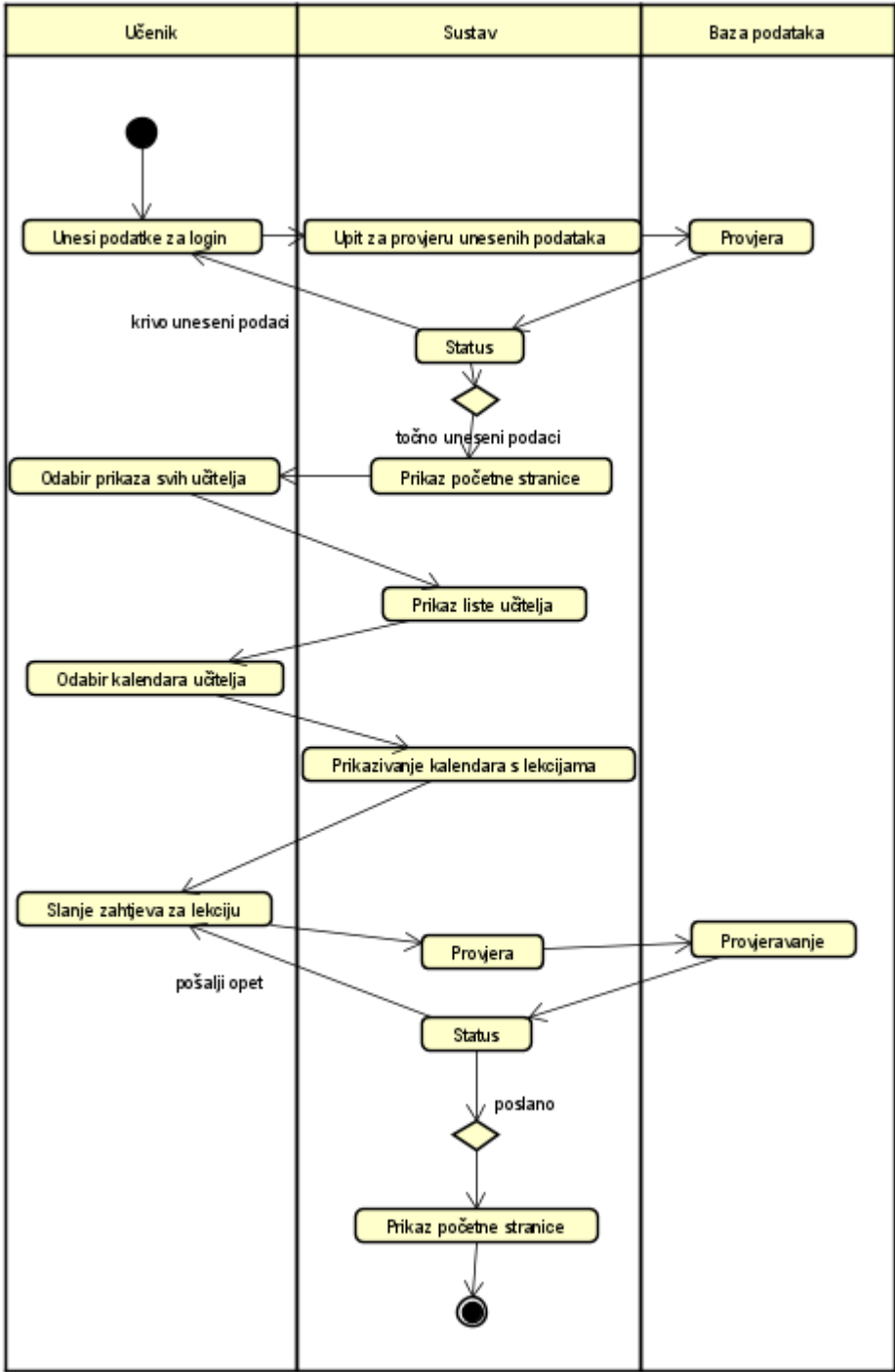
Dijagrami stanja postoje da predoče stanja objekta, te prijelaze iz jednog stanja u drugo na poticaj nekakvog događaja. Na slici imamo prikazanu funkcionalnost i stanja kroz koje učitelj prolazi u slučaju lekcija. Nakon što se prijavi, učitelj može na svoj profil otići i vidjeti sve svoje osobne podatke. Ako odluči nešto promijeniti može samo odabrati "Uredi profil" i moći će unijeti promjene. Na početnoj stranici ukoliko odabere opciju "Arhiva" moći će vidjeti sve prethodne obavljene lekcije. Pritiskom na svoj profil pa na "Kalendar" može unijeti novu lekciju, tako što odabere datum i klikne "Spremi", čime će njegova lekcija biti zakazana. Također može zakazane lekcije pritiskom na gumb "Izbriši" izbrisati i više neće biti aktivne. Učitelj ima prikaz "Zahtjevi" gdje može "Prihvatiti" ili "Odbiti" učenikove zahtjeve za lekciju.



Slika 15: Dijagram stanja.

## Dijagram aktivnosti

Na slici je prikazana funkcionalnost aktivnosti koja pokazuje proces slanja zahtjeva od učenika. Učenik se u sustav prijavljuje sa svojim podacima koje baza provjerava. Ukoliko su netočni onda traži ponovni upis, a ako su ispravno učitavamo početnu stranicu. Učenik odlazi na "Učitelji" gdje mu se prikazuju svi aktivni profesori. Odabirom "Prikaži profil" prikazuje se učiteljev kalendar gdje su vidljive sve lekcije. Učenik odabirom na jednu ili više lekcija može rezervirati, odnosno poslati zahtjev učitelju za tu lekciju. Također se zahtjev provjerava jer možda nije dobro poslan ili se nešto dogodilo prilikom slanja.



Slika 16: Dijagram aktivnosti.

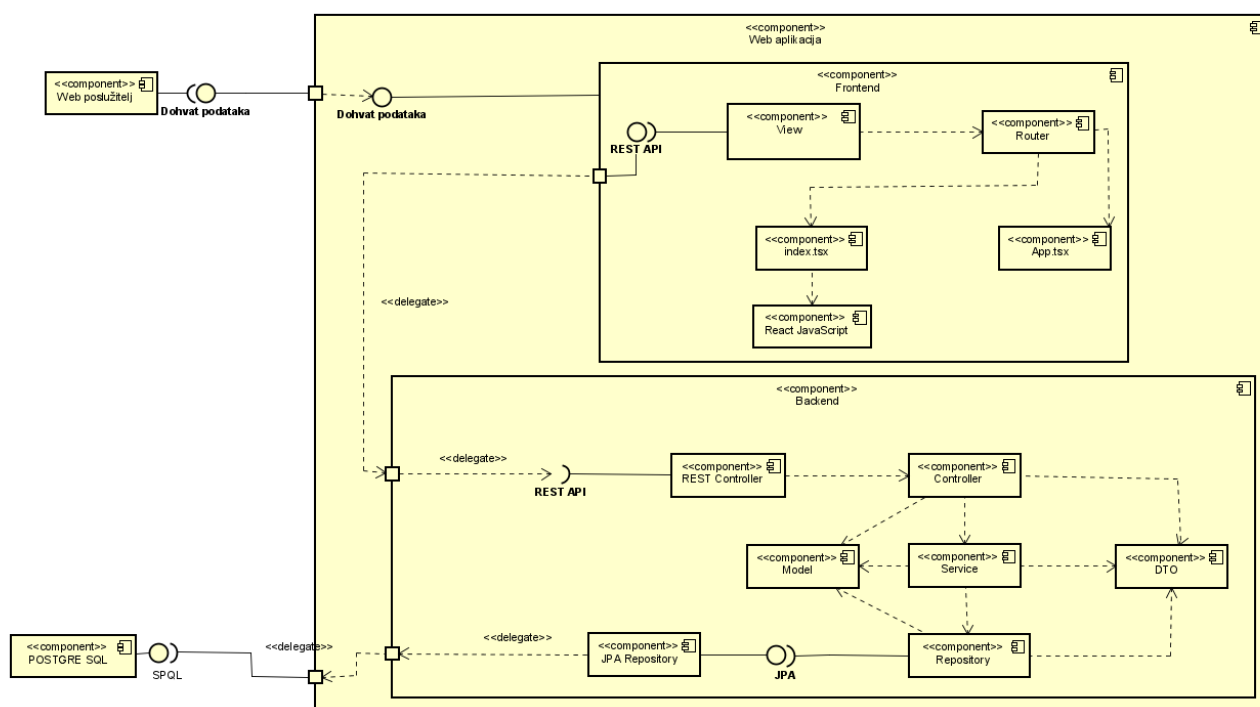
## Dijagram komponentata

Dijagram komponenti sustava ilustrira ključne funkcionalne dijelove aplikacije, njihovu međusobnu povezanost i način komunikacije. Sustav je podijeljen na tri glavne komponente: web poslužitelj, frontend dio i backend dio. Web poslužitelj služi kao ulazna točka za korisnike koji pristupaju aplikaciji putem preglednika. Kroz preglednik korisnici šalju zahtjeve koji se obrađuju u backendu, a rezultati se vraćaju kao odgovori koji se prikazuju u vizualnom obliku na korisničkom sučelju.

Frontend aplikacije odgovoran je za prikaz i interakciju s korisnicima. Ovaj dio aplikacije koristi View komponentu za prikaz podataka i Router za određivanje koje će se stranice prikazati na temelju korisničkih akcija. Komponenta index.jsx predstavlja početnu točku aplikacije koja povezuje sve ostale dijelove frontenda. Glavna uloga App.jsx je upravljanje prikazom grafičkog sučelja i organizacija specifičnih prikaza ovisno o akcijama korisnika. React JavaScript služi kao biblioteka koja omogućuje izgradnju dinamičkih i responzivnih komponenti frontenda. Komunikacija između frontenda i backenda ostvaruje se putem REST API-ja. View komponenta šalje zahtjeve REST API-ju i prima odgovore u JSON formatu, koji se potom prikazuju korisniku.

Backend dio sustava zadužen je za obradu zahtjeva, manipulaciju podacima i interakciju s bazom podataka. Controller komponenta prima zahtjeve iz frontenda, obrađuje ih i šalje odgovore. Service komponenta obrađuje poslovnu logiku aplikacije, prosljeđujući zahtjeve Repository komponenti koja omogućuje pristup podacima iz baze putem JPA sučelja. Podaci se dohvaćaju iz baze podataka PostgreSQL, obrađuju i vraćaju u obliku Data Transfer Object (DTO) objekata. DTO objekti omogućuju efikasnu i sigurnu razmjenu podataka između različitih slojeva aplikacije.

Ovaj dijagram komponenti jasno prikazuje kako sustav funkcionira kroz međusobno povezane komponente koje surađuju putem REST API-ja. Podjela na frontend i backend omogućuje jasnu raspodjelu odgovornosti i učinkovitu obradu podataka, dok korištenje suvremenih tehnologija poput React biblioteke i JPA osigurava pouzdanost i fleksibilnost sustava. Web preglednik korisnicima omogućuje jednostavan pristup sustavu, dok backend osigurava stabilnu i sigurnu obradu podataka te pouzdanu komunikaciju s bazom podataka.



Slika 17: Dijagram komponenti.

## Dijagram razmještaja

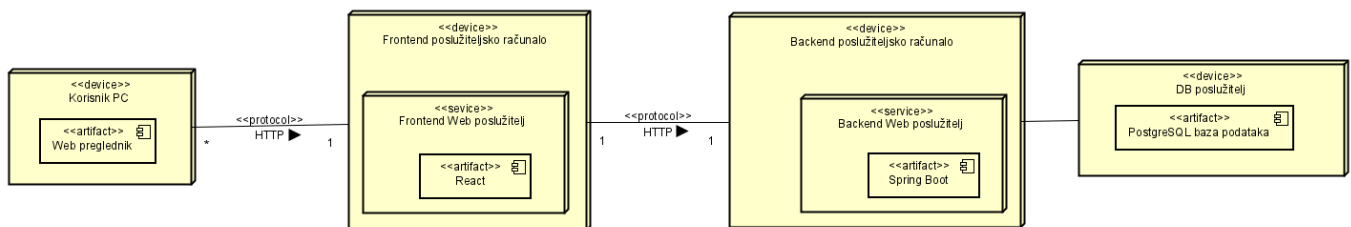
Dijagram razmještaja prikazuje fizičku i virtualnu raspodjelu komponenata unutar infrastrukture sustava te način njihove međusobne komunikacije. Sustav se temelji na klijent-poslužitelj arhitekturi, gdje korisnici pristupaju aplikaciji putem web preglednika, a podaci se razmjenjuju između frontend i backend dijelova putem HTTP protokola. Ovaj dijagram omogućuje bolji pregled načina na koji su raspoređene ključne komponente aplikacije i kako komuniciraju unutar sustava.

U sustavu postoje četiri ključne komponente koje su raspoređene na različitim uređajima i poslužiteljima. Prva komponenta je korisničko računalo, na kojem se nalazi web preglednik koji omogućuje korisniku pristup aplikaciji. Korisnik putem web sučelja šalje zahtjeve prema frontend poslužitelju, koji obrađuje interakcije korisnika i prosljeđuje ih prema backend dijelu sustava.

Druga komponenta predstavlja frontend poslužiteljsko računalo, koje pokreće React aplikaciju. Ova aplikacija služi za prikaz korisničkog sučelja i omogućuje korisnicima interakciju sa sustavom. Frontend poslužitelj komunicira s korisničkim preglednikom putem HTTP protokola te prosljeđuje korisničke zahtjeve backend poslužitelju.

Treća komponenta je backend poslužiteljsko računalo, koje sadrži Spring Boot aplikaciju. Backend je odgovoran za poslovnu logiku sustava i upravljanje podacima. Ovdje se obrađuju zahtjevi koji dolaze s frontenda, provode se potrebne operacije nad podacima te se rezultati vraćaju prema korisniku putem REST API-ja. Backend poslužitelj također komunicira s bazom podataka kako bi dohvaćao, ažurirao ili brisao podatke prema potrebi.

Četvrta i posljednja komponenta je DB poslužiteljsko računalo, koje pokreće PostgreSQL bazu podataka. Ova komponenta služi za sigurno pohranjivanje podataka i omogućuje backend sustavu pristup podacima kroz definirane upite. Backend komunicira s bazom podataka direktno, osiguravajući da su podaci pohranjeni i dohvaćeni na siguran i efikasan način.



Slika 18: Dijagram razmještaja.

# Ispitivanje programskog rješenja

---

## Ispitivanje komponenti

### 1. Trenutni korisnik

Ulaz: Web Stranica: `http://localhost:5173` , Cookie = cookie koji dobijemo nakon što se ulogiramo s korisničkim računom.

---

- **Koraci:**

1. Pokrenuti aplikaciju
2. Otvoriti web stranicu aplikacije
3. Ulogirati se u aplikaciju
4. Dobiveni cookie spremiti u Postman
5. Isprobati GET endpoint `/trenutnikorisnik`

- **Očekivani izlaz:** `{ "id": 1, "name": "Tomo", "email": "tomislavk4444@gmail.com", "picture": "http://example.com/johndoe.jpg", "role": "ROLE_ADMIN" }.`
- 

### 2. Ažuriranje korisnika ( ovaj korak se nastavlja na prijasnji)

Ulaz: Web Stranica: `http://localhost:5173` , Cookie = cookie koji dobijemo nakon što se ulogiramo s korisničkim računom. Name: novo ime, Slika: novi url slike

`{ "picture": null, "name": "Tomo Kragi" }`

---

- **Koraci:**

1. Poslati POST request za ažuriranje podataka
2. Unijeti novo ime i novu sliku ( ime / slika smiju biti null)
3. Provjeriti jesu li spremljene promijene

- **Očekivani izlaz:** `{ "id": 1, "name": "Tomo Kragi", "email": "tomislavk4444@gmail.com", "picture": null, "role": "ROLE_ADMIN" }`
- 

### 3. Vraćanje podataka o korisniku po ID-u

Ulaz: Web Stranica: `http://localhost:5173` , Cookie = cookie koji dobijemo nakon što se ulogiramo s korisničkim računom.

---

- **Koraci:**

1. Poslati GET request za dohvat podataka o korisniku
2. ID = 20
3. Izvješće o pogrešci : Not found

4. Poslati GET request za dohvat podataka o korisniku
5. ID = 5
6. Dobiti podatke o postojećem korisniku

- **Očekivani izlaz:** "

```
{ "timestamp": "2025-01-17T04:42:30.819+00:00", "status": 404, "error": "Not Found", "path": "/korisnici/20" }
```

```
{ "id": 5, "name": "Marija Doe", "email": "Marija.doe@example.com", "picture":  
"http://example.com/johndoe.jpg", "role": "ROLE_ADMIN" }."
```

#### 4. Interakcija s Postmanom bez validnog cookie-a

Ulaz: Web Stranica: <http://localhost:5173> , Cookie = cookie koji dobijemo nakon što se ulogiramo s korisničkim računom.

---

- **Koraci:**

1. Pokrenuti aplikaciju
2. Otvoriti web stranicu aplikacije
3. Ulogirati se u aplikaciju
4. Dobiveni cookie nećemo spremiti u Postman
5. Isprobati GET endpoint /trenutniucitelj

- **Očekivani izlaz:** "

```
<!DOCTYPE html>  
<html lang="en">  
  
<head>  
  <meta charset="utf-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-  
fit=no">  
  <meta name="description" content="">  
  <meta name="author" content="">  
  <title>Please sign in</title>  
  <link href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0-  
beta/css/bootstrap.min.css" rel="stylesheet"  
  
  integrity="sha384-/Y6pD6FV/Vv2HJnA6t+vs1U6fwYXjCFtcEpHbNJ0lyAFsXTsjBbfaDjzALeQsN6M  
" crossorigin="anonymous">  
  <link href="https://getbootstrap.com/docs/4.0/examples/signin/signin.css"  
rel="stylesheet"  
  integrity="sha384-  
o0E/3m0LUMPub4kaC09mrdEhIc+e3exm4x0GxAmuFXhBNF4hcg/6MiAXAf5p0P56"  
crossorigin="anonymous" />  
</head>  
  
<body>  
  <div class="container">  
    <h2 class="form-signin-heading">Login with OAuth 2.0</h2>
```

```
<table class="table table-striped">
  <tr>
    <td><a href="/oauth2/authorization/github">GitHub</a></td>
  </tr>
  <tr>
    <td><a href="/oauth2/authorization/google">Google</a></td>
  </tr>
</table>
</div>
</body>

</html>."
```

## 5. Bad request

Ulaz: Web Stranica: <http://localhost:5173> , Cookie = cookie koji dobijemo nakon što se ulogiramo s korisničkim računom. { "godinelskustva": 10, "kvalifikacija": "Dokorat", "stilPoducavanja": "Gramatičko\_prijevodna\_metodatt", "satnica": 50.0, "jezici": ["Kineski", "Engleski", "Hindski"] }

---

- **Koraci:**

1. Pokrenuti aplikaciju
2. Poslati POST request za ažuriranje učitelja
3. Nažalost postoji tipfeler u imenu stila podučavanja
4. Dobijemo poruku o iznimci

- **Očekivani izlaz:** " { "timestamp": "2025-01-17T06:33:36.876+00:00", "status": 400, "error": "Bad Request", "path": "/azurirajucitelja" }."

## 6. Internal Server Error

Ulaz: Web Stranica: <http://localhost:5173> , Cookie = cookie koji dobijemo nakon što se ulogiramo s korisničkim računom.

---

- **Koraci:**

1. Pokrenuti aplikaciju
2. Otvoriti web stranicu aplikacije
3. Ulogirati se u aplikaciju
4. Ažurirati podatke o učeniku
5. Isprobati POST endpoint /azurirajucenika
6. Vrijednost jezika je null
7. Izbacuje nam grešku

- **Očekivani izlaz:** "{ "timestamp": "2025-01-17T06:52:53.916+00:00", "status": 500, "error": "Internal Server Error", "path": "/azurirajucenika" }."

## Ispitivanje sustava

## Primjeri ispitivanja sa Seleniumom:

---

### 1. Formular za prijavu:

Ulaz: Web Stranica: <https://progici2front-62a5e06d95e8.herokuapp.com> , Email = "Vas email", Lozinka = "Dobra lozinka".

---

- **Koraci:**

1. Otvoriti aplikaciju na zadanoj adresi
2. Kliknuti gumb za prijavu
3. Odabrati prijavu preko google-a
4. Unijeti email i lozinku
5. Nastaviti dalje sa prijavom
6. Korisnik je preusmjeren na početnu stranicu

- **Očekivani izlaz:** "Prijava uspješna."

► PRIKAŽI CIJELI KOD

```
package UI;

import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.logging.LogEntries;
import org.openqa.selenium.logging.LogEntry;
import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.Keys;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

public class TryOut {
    public static void main(String[] args) {

        // Step 1: Set the path to the ChromeDriver executable
        System.setProperty("webdriver.chrome.driver", "C:\\\\131.0.6778.69
chromedriver-win32\\\\chromedriver-win32\\\\chromedriver.exe"); // Update with the
actual path

        // Step 2: Initialize the WebDriver (ChromeDriver)
        ChromeOptions options = new ChromeOptions();
        options.setExperimentalOption("debuggerAddress", "localhost:9222");
        ChromeDriver driver = new ChromeDriver(options);

        driver.get("https://progici2front-62a5e06d95e8.herokuapp.com/");
        driver.manage().timeouts().implicitlyWait(Duration.ofMillis(5000));
        driver.getTitle();
    }
}
```

```

//try out Login

driver.findElement(By.xpath("/html/body/div/nav/div/div[2]/a/button")).click();
try {
    Thread.sleep(2000); // Wait for 2 seconds
} catch (InterruptedException e) {
    e.printStackTrace();
}
//try out Login

driver.findElement(By.xpath("/html/body/div/div/div/div[3]/a[2]")).click();
try {
    Thread.sleep(2000); // Wait for 2 seconds
} catch (InterruptedException e) {
    e.printStackTrace();
}
// Let's Login
WebElement emailField =
driver.findElement(By.xpath("/html/body/div[1]/div[1]/div[2]/c-
wiz/div/div[2]/div/div/div[1]/form/span/section/div/div/div[1]/div/div[1]/div/div[
1]/input"));

emailField.clear(); // Clear any pre-filled email
emailField.click(); // Make sure the field is focused
emailField.sendKeys("privatniemail"); // First part of the email
//emailField.sendKeys(Keys.SHIFT); // Hold Shift for '@'
emailField.sendKeys("@"); // Type '@' symbol
emailField.sendKeys("gmail.com"); // Rest of the email
try { Thread.sleep(2000); } catch (InterruptedException e) {
e.printStackTrace(); }
try {
    Thread.sleep(2000); // Wait for 2 seconds
} catch (InterruptedException e) {
    e.printStackTrace();
}
driver.findElement(By.xpath("/html/body/div[1]/div[1]/div[2]/c-
wiz/div/div[3]/div/div[1]/div/div/button/span")).click();
try {
    Thread.sleep(2000); // Wait for 2 seconds
} catch (InterruptedException e) {
    e.printStackTrace();
}
//Lets Login
driver.findElement(By.xpath("/html/body/div[1]/div[1]/div[2]/c-
wiz/div/div[2]/div/div/div[1]/form/span/section[2]/div/div/div[1]/div[1]/div/div/d
iv/div/div[1]/div/div[1]/input")).sendKeys("privatnaZaporka");
try {
    Thread.sleep(2000); // Wait for 2 seconds
} catch (InterruptedException e) {
    e.printStackTrace();
}
driver.findElement(By.xpath("/html/body/div[1]/div[1]/div[2]/c-
wiz/div/div[3]/div/div[1]/div/div/button/span")).click();

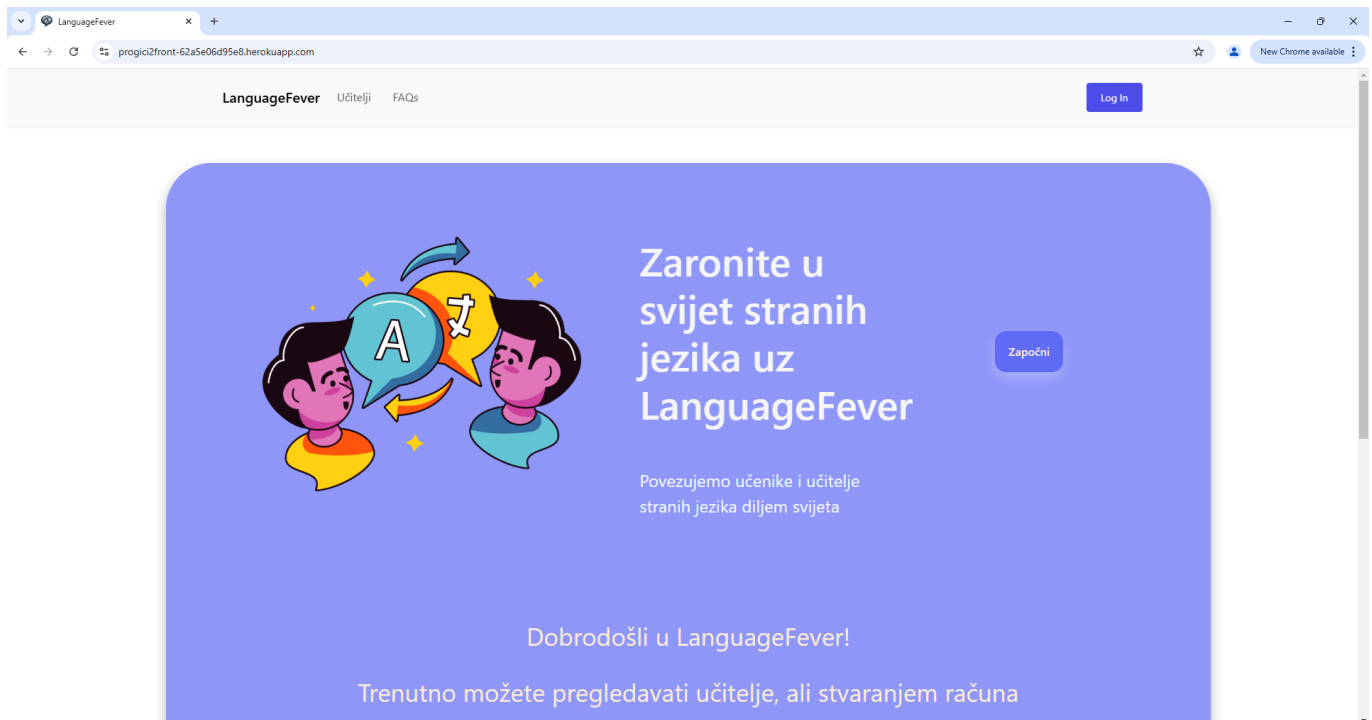
```

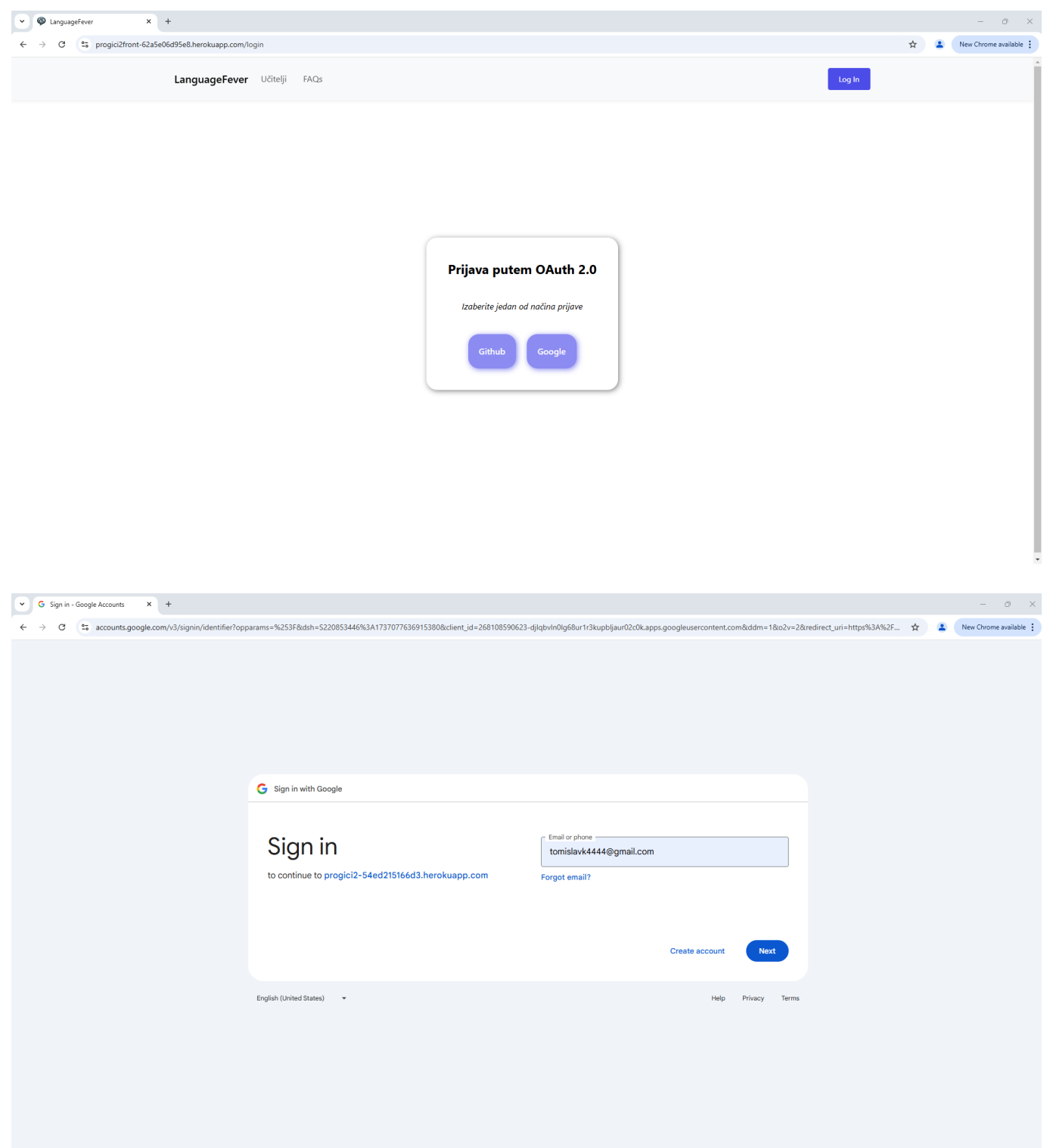
```
try {
    Thread.sleep(2000); // Wait for 2 seconds
} catch (InterruptedException e) {
    e.printStackTrace();
}
//driver.findElement(By.xpath("/html/body/div[1]/div[1]/div[2]/c-
wiz/div/div[3]/div/div/div[2]/div/div/button/span")).click();
try {
    Thread.sleep(2000); // Wait for 2 seconds
} catch (InterruptedException e) {
    e.printStackTrace();
}
// Dobijanje logova iz browsera
LogEntries logs = driver.manage().logs().get("browser");
for (LogEntry entry : logs) {
    System.out.println(entry.getMessage());
}

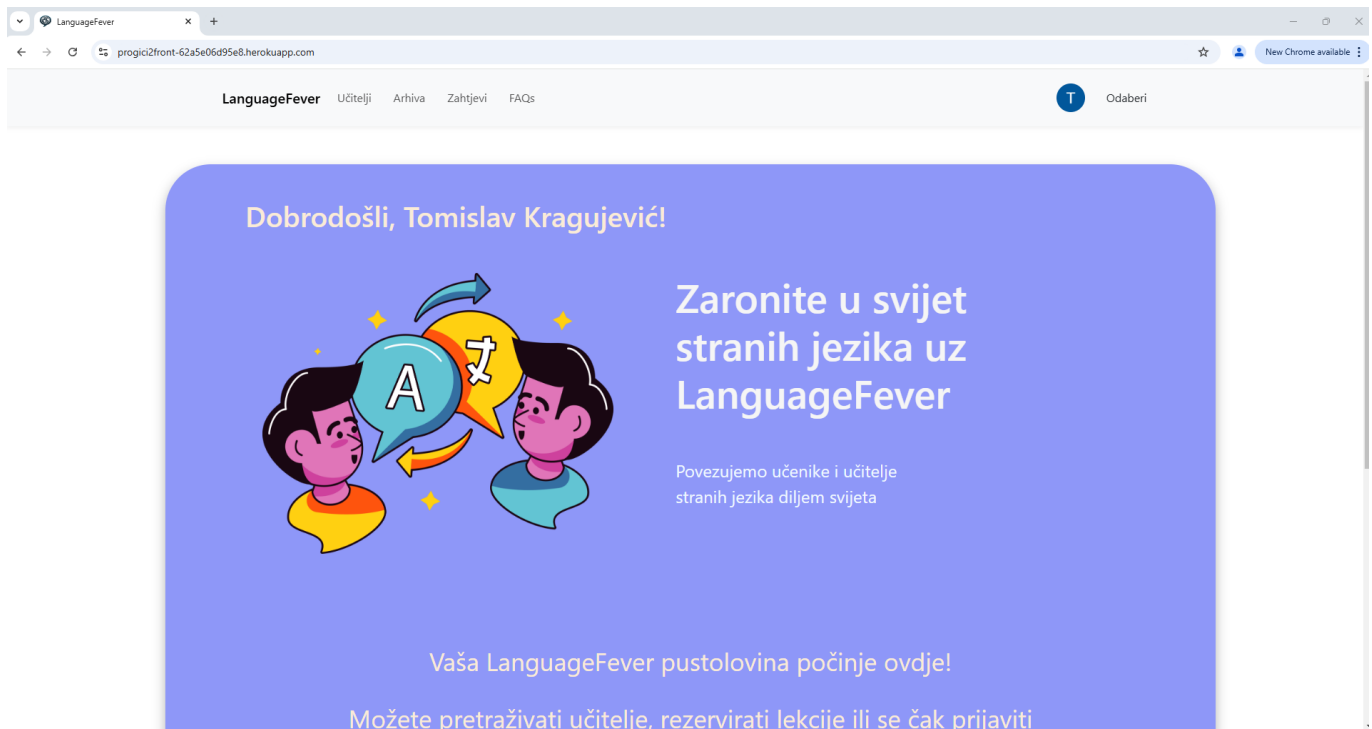
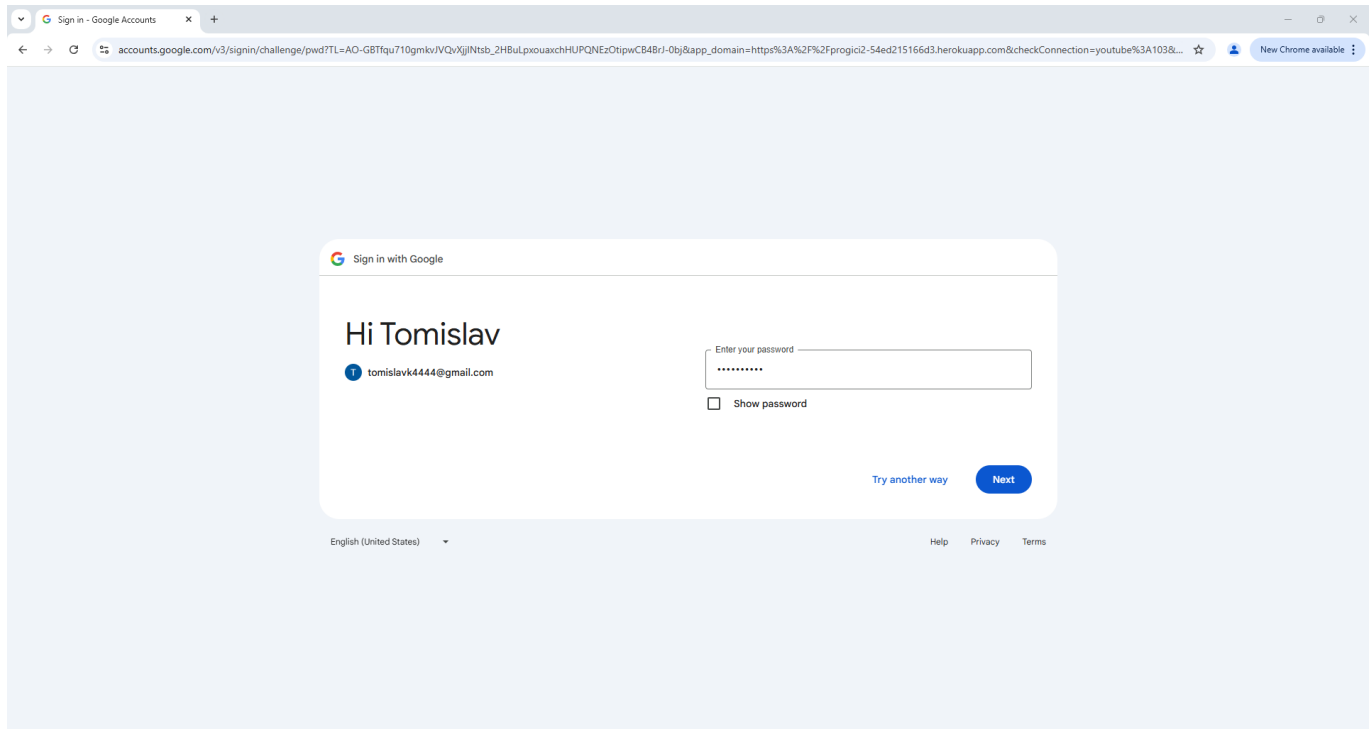
}

}
```

## ► PRIKAŽI SLIKE







---

## 2. Rubni uvjet – nevažeća lozinka:

Ulaz: Web Stranica: <https://progici2front-62a5e06d95e8.herokuapp.com> , Email = "Vas email", Lozinka = "kriva lozinka".

---

- **Koraci:**

1. Otvoriti aplikaciju na zadanoj adresi
2. Kliknuti gumb za prijavu
3. Odabrati prijavu preko google-a
4. Unijeti email i lozinku

5. Kliknuti na "Prijava".
  6. Provjeriti je li prikazana poruka o grešci.
- **Očekivani izlaz:** "Pogrešno korisničko ime ili lozinka."

► PRIKAŽI CIJELI KOD

```
package UI;

import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.Keys;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

public class TryOutFalse {
    public static void main(String[] args) {

        // Step 1: Set the path to the ChromeDriver executable
        System.setProperty("webdriver.chrome.driver", "C:\\\\131.0.6778.69
chromedriver-win32\\\\chromedriver-win32\\\\chromedriver.exe"); // Update with the
actual path

        // Step 2: Initialize the WebDriver (ChromeDriver)
        ChromeOptions options = new ChromeOptions();
        options.setExperimentalOption("debuggerAddress", "localhost:9222");
        ChromeDriver driver = new ChromeDriver(options);

        driver.get("https://progici2front-62a5e06d95e8.herokuapp.com/");
        driver.manage().timeouts().implicitlyWait(Duration.ofMillis(5000));
        driver.getTitle();

        //try out Login

        driver.findElement(By.xpath("/html/body/div/nav/div/div[2]/a/button")).click();
        try {
            Thread.sleep(2000); // Wait for 2 seconds
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        //try out Login

        driver.findElement(By.xpath("/html/body/div/div/div/div[3]/a[2]")).click();
        try {
            Thread.sleep(2000); // Wait for 2 seconds
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

```

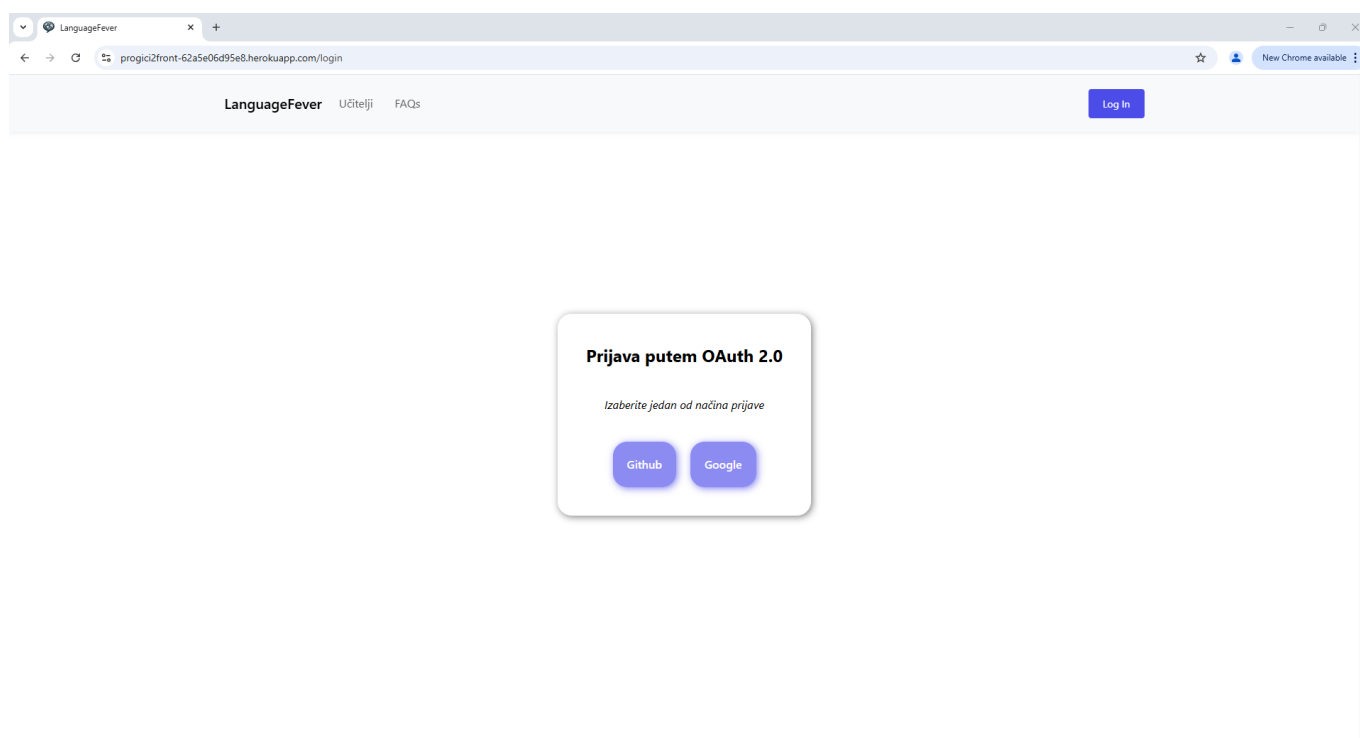
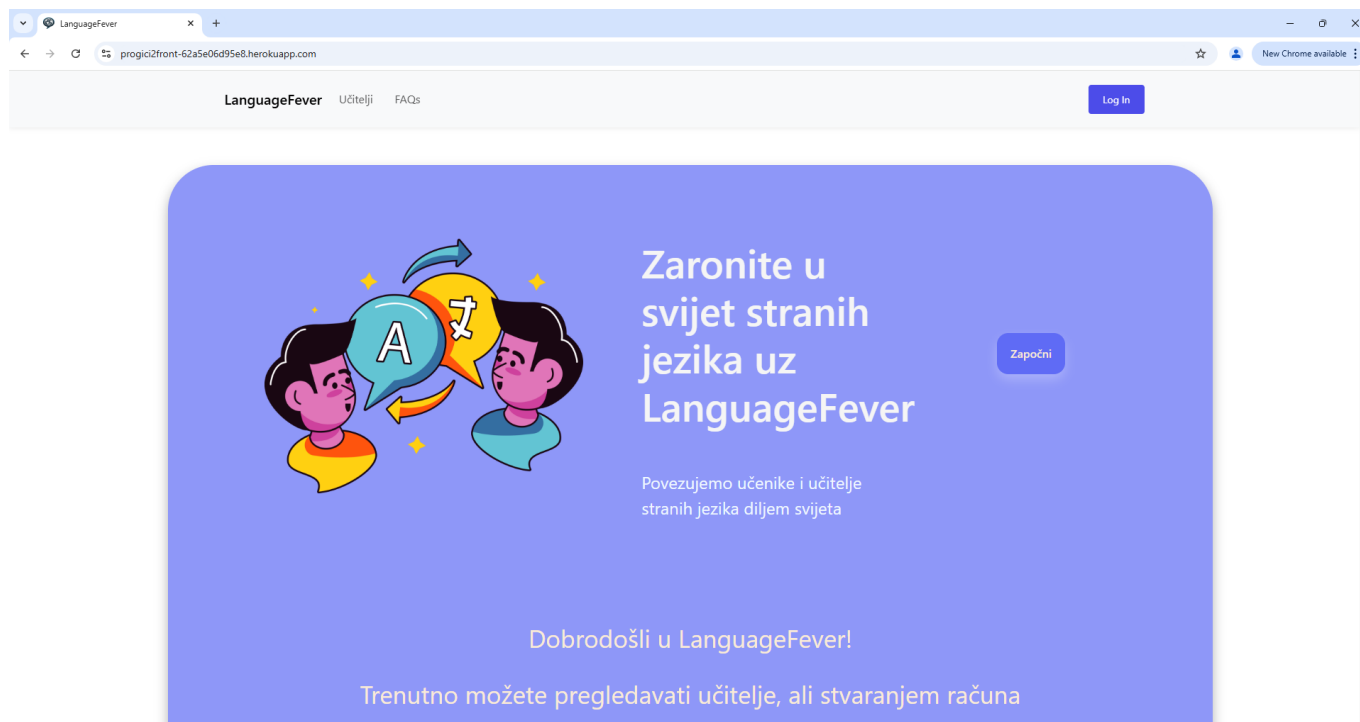
        // Let's Login
        WebElement emailField =
driver.findElement(By.xpath("/html/body/div[1]/div[1]/div[2]/c-
wiz/div/div[2]/div/div/div[1]/form/span/section/div/div/div[1]/div/div[1]/div/div[
1]/input"));

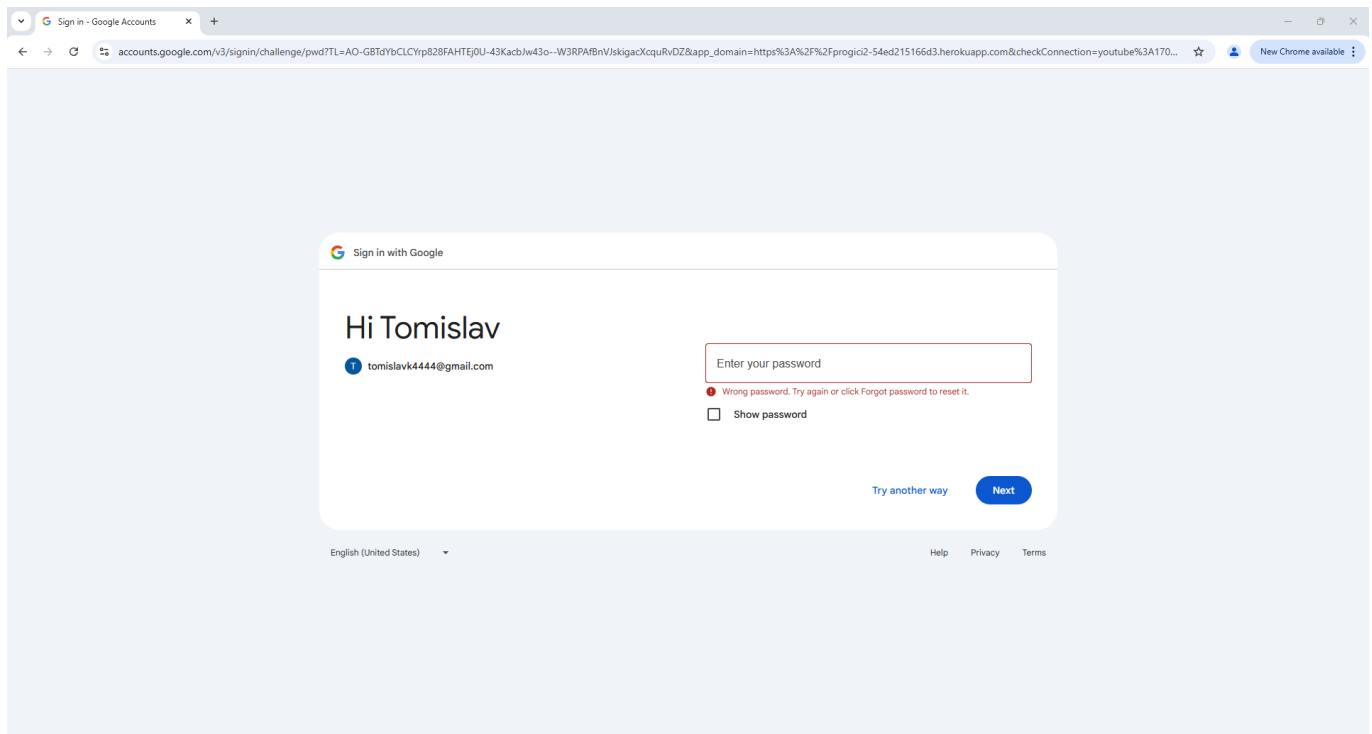
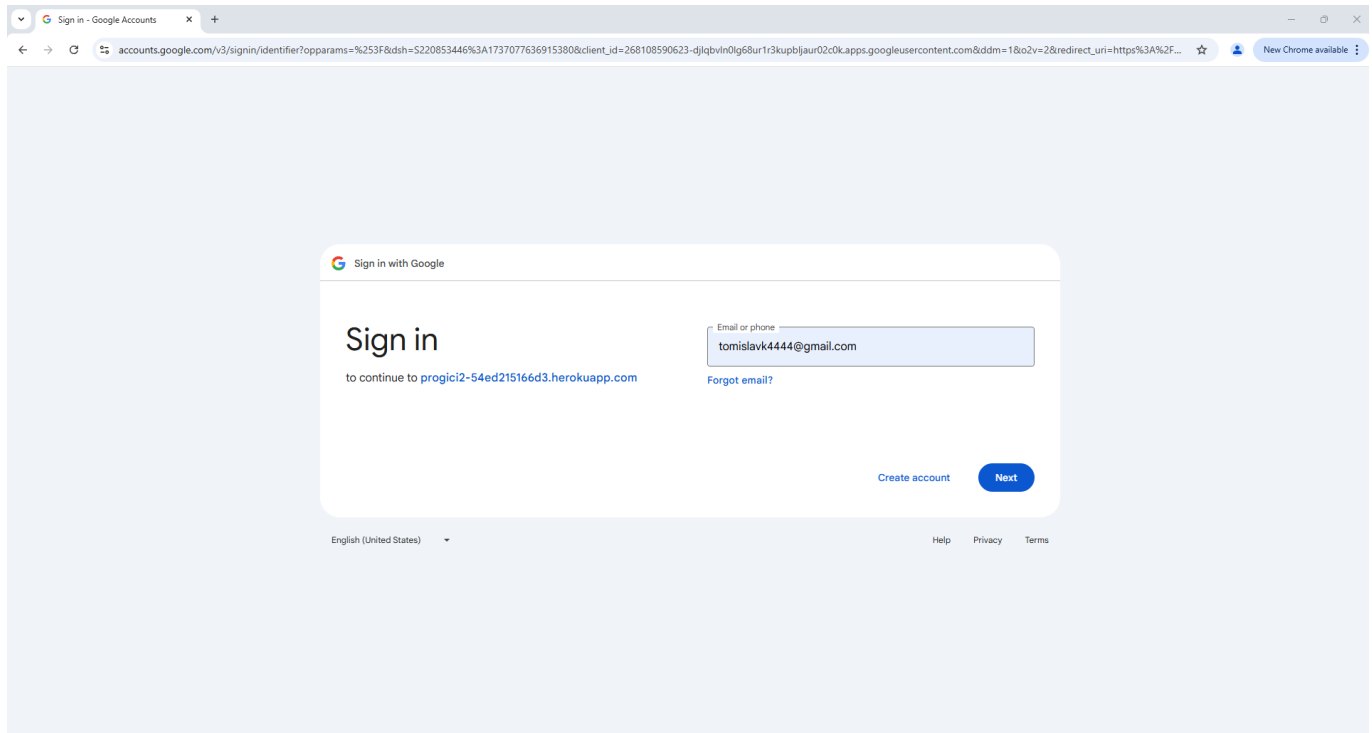
        emailField.clear(); // Clear any pre-filled email
        emailField.click(); // Make sure the field is focused
        emailField.sendKeys("tomislavk4444"); // First part of the email
        //emailField.sendKeys(Keys.SHIFT); // Hold Shift for '@'
        emailField.sendKeys("@"); // Type '@' symbol
        emailField.sendKeys("gmail.com"); // Rest of the email
        try { Thread.sleep(2000); } catch (InterruptedException e) {
e.printStackTrace(); }
        try {
            Thread.sleep(2000); // Wait for 2 seconds
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        driver.findElement(By.xpath("/html/body/div[1]/div[1]/div[2]/c-
wiz/div/div[3]/div/div/div[1]/div/div/button/span")).click();
        try {
            Thread.sleep(2000); // Wait for 2 seconds
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        //Lets Login
        driver.findElement(By.xpath("/html/body/div[1]/div[1]/div[2]/c-
wiz/div/div[2]/div/div/div[1]/form/span/section[2]/div/div/div[1]/div[1]/div/div/d
iv/div/div[1]/div/div[1]/input")).sendKeys("Kriva");
        try {
            Thread.sleep(2000); // Wait for 2 seconds
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        driver.findElement(By.xpath("/html/body/div[1]/div[1]/div[2]/c-
wiz/div/div[3]/div/div/div[1]/div/div/button/span")).click();
        try {
            Thread.sleep(2000); // Wait for 2 seconds
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        driver.findElement(By.xpath("/html/body/div[1]/div[1]/div[2]/c-
wiz/div/div[3]/div/div/div[2]/div/div/button/span")).click();
        try {
            Thread.sleep(2000); // Wait for 2 seconds
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

```

}

## ► PRIKAŽI SLIKE





#### ► console

Exception in thread "main" org.openqa.selenium.NoSuchElementException: no such element: Unable to locate element: {"method":"xpath","selector":"/html/body/div[1]/div[1]/div[2]/c-wiz/div/div[3]/div/div/div[2]/div/div/button/span"} (Session info: chrome=131.0.6778.86) For documentation on this error, please visit: <https://www.selenium.dev/documentation/webdriver/troubleshooting/errors#no-such-element-exception> Build info: version: '4.27.0', revision: 'd6e718d134' System info: os.name: 'Windows 11', os.arch: 'amd64', os.version: '10.0', java.version: '17.0.11' Driver info: org.openqa.selenium.chrome.ChromeDriver Command: [893aac06ccb3a401c616da98aa91d6f4, findElement {using=xpath, value=/html/body/div[1]/div[1]/div[2]/c-wiz/div/div[3]/div/div/div[2]/div/div/button/span}] Capabilities {acceptInsecureCerts: false, browserName: chrome, browserVersion: 131.0.6778.86, chrome: {chromedriverVersion: 131.0.6778.69 (77e2244bbcd...), fedcm:accounts: true, goog:chromeOptions:

```
{debuggerAddress: localhost:9222}, pageLoadStrategy: normal, platformName: any, proxy: Proxy(), se:cdp:
ws://localhost:9222/devtool..., se:cdpVersion: 131.0.6778.86, setWindowRect: false, strictFileInteractability:
false, timeouts: {implicit: 0, pageLoad: 300000, script: 30000}, unhandledPromptBehavior: dismiss and notify,
webauthn:extension:credBlob: true, webauthn:extension:largeBlob: true, webauthn:extension:minPinLength:
true, webauthn:extension:prf: true, webauthn:virtualAuthenticators: true} Session ID:
893aac06ccb3a401c616da98aa91d6f4 at
java.base/jdk.internal.reflect.NativeConstructorAccessorImpl.newInstance0(Native Method) at
java.base/jdk.internal.reflect.NativeConstructorAccessorImpl.newInstance(NativeConstructorAccessorImpl.java:
77) at
java.base/jdk.internal.reflect.DelegatingConstructorAccessorImpl.newInstance(DelegatingConstructorAccessorI
mpl.java:45) at java.base/java.lang.reflect.Constructor.newInstanceWithCaller(Constructor.java:499) at
java.base/java.lang.reflect.Constructor.newInstance(Constructor.java:480) at
org.openqa.selenium.remote.ErrorCodec.decode(ErrorCodec.java:167) at
org.openqa.selenium.remote.codec.w3c.W3CHttpResponseCodec.decode(W3CHttpResponseCodec.java:138)
at org.openqa.selenium.remote.codec.w3c.W3CHttpResponseCodec.decode(W3CHttpResponseCodec.java:50)
at org.openqa.selenium.remote.HttpCommandExecutor.execute(HttpCommandExecutor.java:190) at
org.openqa.selenium.remote.service.DriverCommandExecutor.invokeExecute(DriverCommandExecutor.java:21
6) at org.openqa.selenium.remote.service.DriverCommandExecutor.execute(DriverCommandExecutor.java:174)
at org.openqa.selenium.remote.RemoteWebDriver.execute(RemoteWebDriver.java:545) at
org.openqa.selenium.remote.ElementLocation$ElementFinder$2.findElement(ElementLocation.java:165) at
org.openqa.selenium.remote.ElementLocation.findElement(ElementLocation.java:59) at
org.openqa.selenium.remote.RemoteWebDriver.findElement(RemoteWebDriver.java:368) at
org.openqa.selenium.remote.RemoteWebDriver.findElement(RemoteWebDriver.java:362) at
UI.TryOutFalse.main(TryOutFalse.java:77)
```

---

### 3. Uređivanje profila:

Ulaz: Web Stranica: <https://progici2front-62a5e06d95e8.herokuapp.com> , Odabrani jezik: Španjolski

---

- **Koraci:**

1. Otvoriti aplikaciju na zadanoj adresi
2. Kliknuti na profil
3. Odabrati uredi profil
4. Kliknuti na polje za dodavanje jezika
5. Odabrati željeni jezik
6. Spremiti ažurirane podatke
7. Vratiti se na popis učitelja

- **Očekivani izlaz:** "Podatci uspješno ažurirani."

► PRIKAŽI CIJELI KOD

```
package UI;

import java.time.Duration;
```

```
import org.openqa.selenium.By;
import org.openqa.selenium.Keys;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.interactions.Actions;

public class Profil3 {
    public static void main(String[] args) {

        // Step 1: Set the path to the ChromeDriver executable
        System.setProperty("webdriver.chrome.driver", "C:\\131.0.6778.69
chromedriver-win32\\chromedriver-win32\\chromedriver.exe"); // Update with the
actual path

        // Step 2: Initialize the WebDriver (ChromeDriver)
        ChromeOptions options = new ChromeOptions();
        options.setExperimentalOption("debuggerAddress", "localhost:9222");
        ChromeDriver driver = new ChromeDriver(options);

        driver.get("https://progici2front-62a5e06d95e8.herokuapp.com/");
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));

        try {
            // Navigate to FAQ

driver.findElement(By.xpath("/html/body/div/nav/div/div[2]/div[2]/ul/li[1]/a/butto
n")).click();
            Thread.sleep(2000); // Wait for 2 seconds

            // Navigate to Language Fever

driver.findElement(By.xpath("/html/body/div/div[2]/div/ul/li[2]")).click();
            Thread.sleep(2000); // Wait for 2 seconds

            // Open the dropdown
            WebElement dropdown =
driver.findElement(By.xpath("/html/body/div/div[2]/div/div[2]/div/form/div[1]/div[
1]/div/div[1]/div[1]/div[2]"));
            dropdown.click(); // Open the dropdown
            Thread.sleep(2000); // Wait for options to load

            // Select "Španjolski" option
            WebElement spanjolskiOption =
driver.findElement(By.xpath("//div[text()='Španjolski']"));
            spanjolskiOption.click(); // Click on the option
            Thread.sleep(1000); // Allow selection to process

            // Try out Saving

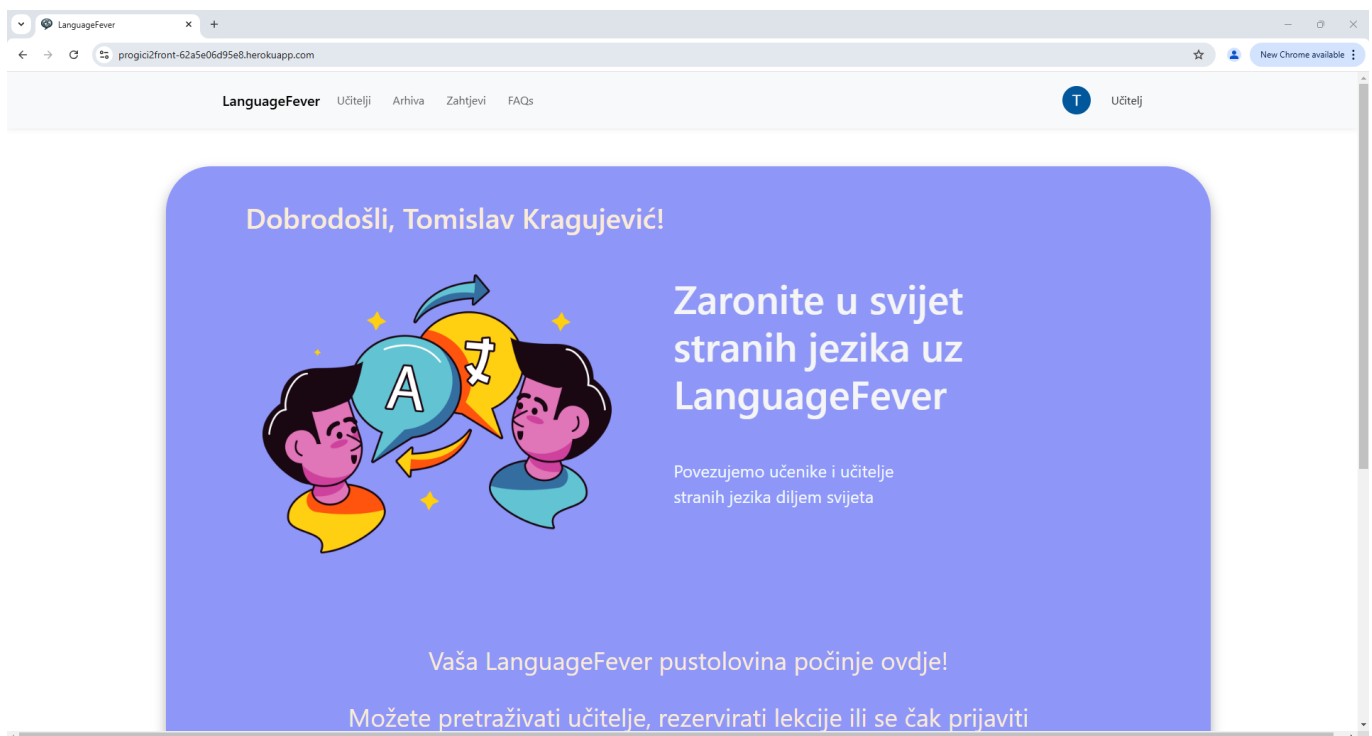
driver.findElement(By.xpath("/html/body/div/div[2]/div/div[2]/div/form/div[2]/butt
on")).click();
            Thread.sleep(2000); // Wait for 2 seconds
```

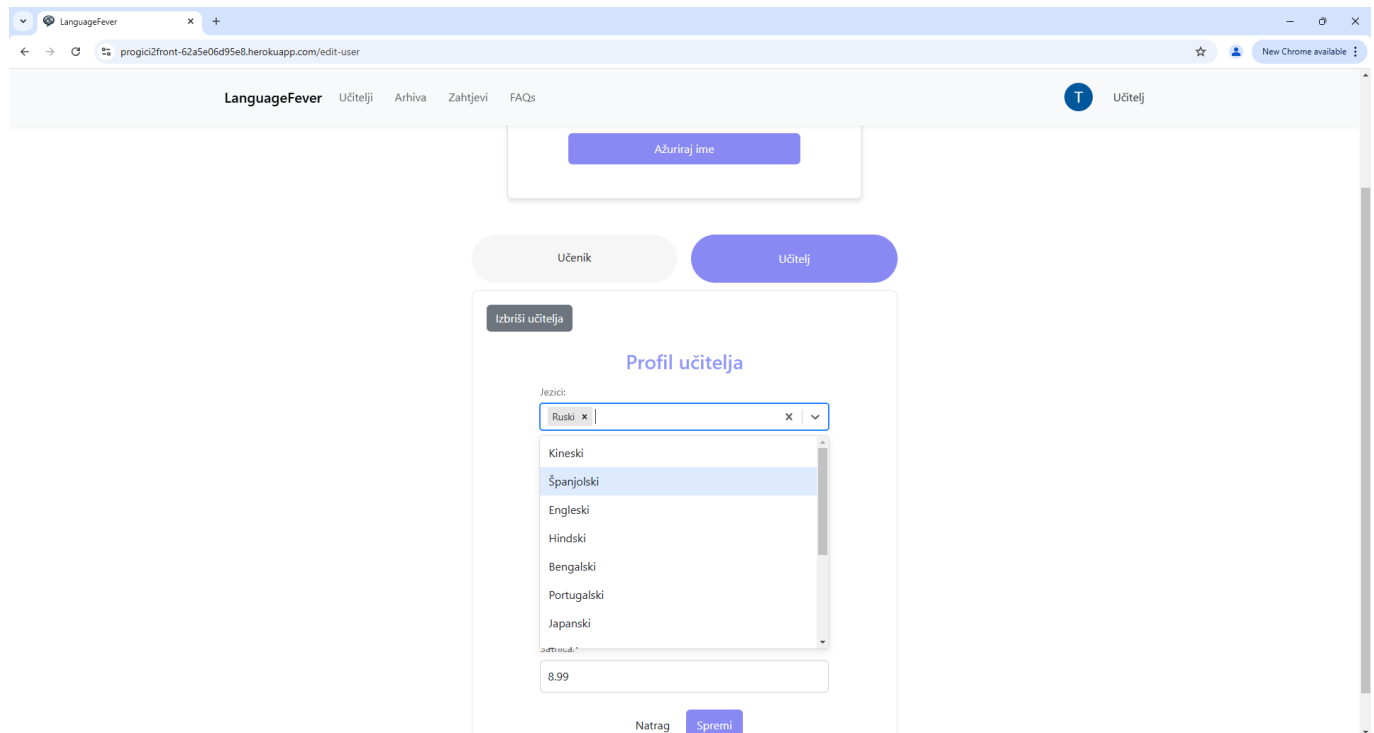
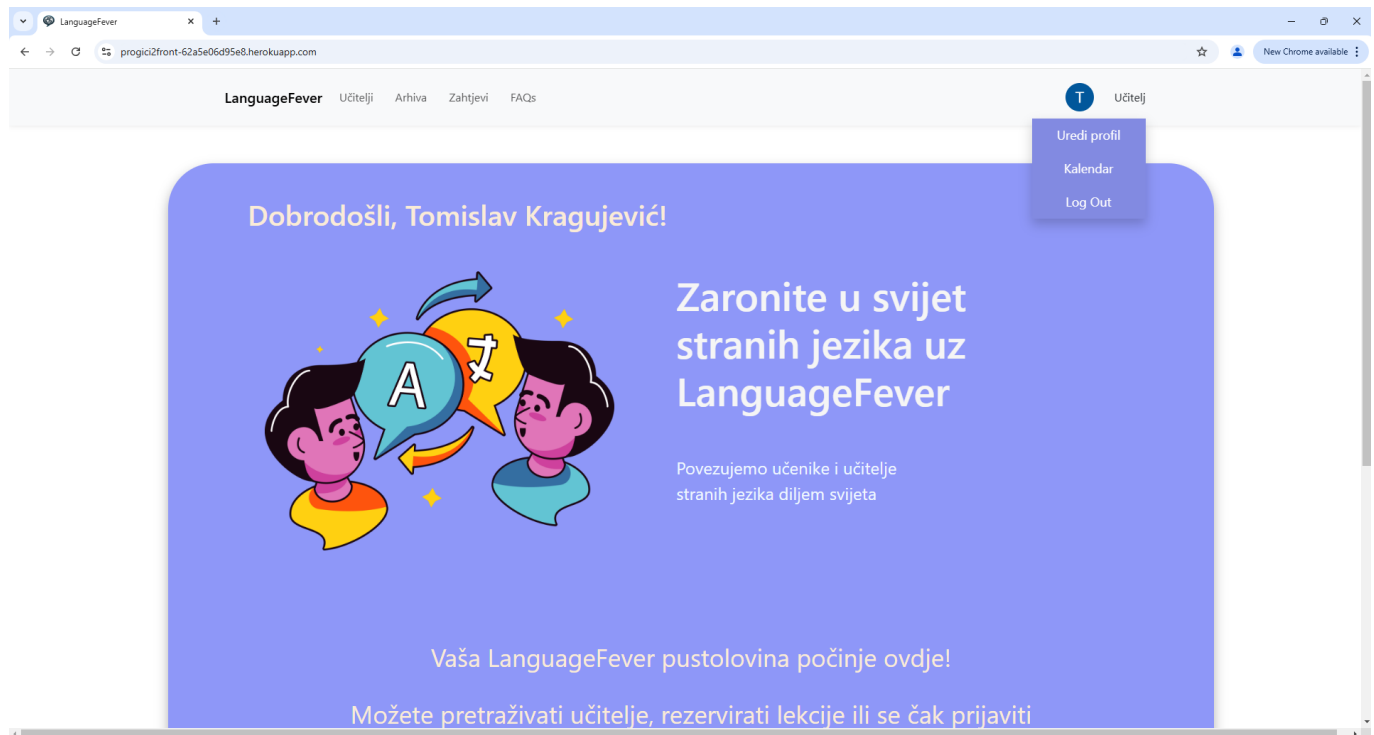
```
// Try out Ucitelji

driver.findElement(By.xpath("/html/body/div/nav/div/div[1]/a[1]")).click();
Thread.sleep(2000); // Wait for 2 seconds

} catch (Exception e) {
    e.printStackTrace();
} finally {
    // Close the browser
}
}
```

## ► PRIKAŽI SLIKE





LanguageFever Učitelji Arhiva Zahtjevi FAQs

T Učitelj

Učenik Učitelj

Izbrisi učitelja

### Profil učitelja

Jezici: Španjolski x Ruski x

Stil podučavanja: Sugestopedija

Kvalifikacije: Magistarski studij

Godine iskustva: 7

Satnica: 8.99

Natrag Spremi

✓ Podaci o učitelju su uspješno ažurirani

LanguageFever Učitelji Arhiva Zahtjevi FAQs

T Učitelj

Filter Sortiranje

**Josip Pavić**  
Jezici: Španjolski  
Ocjena: 0.00

Prikaži detalje [Prikaži profil](#)

**Dario Sučevac**  
Jezici: Engleski, Ruski, Njemački  
Ocjena: 5.00

Prikaži detalje [Prikaži profil](#)

**Language Fever**  
Jezici: Engleski, Hindijski, Japanski  
Ocjena: 0.00

Prikaži detalje [Prikaži profil](#)

**Fran Bistrović**  
Jezici: Španjolski, Korejski, Portugalski  
Ocjena: 0.00

Prikaži detalje [Prikaži profil](#)

#### 4. Sortiranje učitelja

Ulaz: Web Stranica: <https://progici2front-62a5e06d95e8.herokuapp.com> , MinCijena = "6", MaxCijena = "17".

- Koraci:**

1. Otvoriti aplikaciju na zadanoj adresi
2. Kliknuti popis svih ucitelja
3. Odabrati gumb za sortiranje
4. Unijeti min i max cijene
5. Kliknuti izvan prozora

6. Korisnik je dobio sortirani popis učitelja

- **Očekivani izlaz:** "Sortirani popis ucitelja."

► PRIKAŽI CIJELI KOD

```
package UI;

import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.Keys;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

public class Sorting {
    public static void main(String[] args) {

        // Step 1: Set the path to the ChromeDriver executable
        System.setProperty("webdriver.chrome.driver", "C:\\\\131.0.6778.69
chromedriver-win32\\\\chromedriver-win32\\\\chromedriver.exe"); // Update with the
actual path

        // Step 2: Initialize the WebDriver (ChromeDriver)
        ChromeOptions options = new ChromeOptions();
        options.setExperimentalOption("debuggerAddress", "localhost:9222");
        ChromeDriver driver = new ChromeDriver(options);

        driver.get("https://progici2front-62a5e06d95e8.herokuapp.com/");
        driver.manage().timeouts().implicitlyWait(Duration.ofMillis(5000));
        driver.getTitle();
        //Try out Ucitelji

        driver.findElement(By.xpath("/html/body/div/nav/div/div[1]/a[1]")).click();
        try {
            Thread.sleep(2000); // Wait for 2 seconds
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        //Try out Filter

        driver.findElement(By.xpath("/html/body/div/div/div[1]/div/div/button[1]")).click(
);
        try {
            Thread.sleep(2000); // Wait for 2 seconds
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        //Min price
```

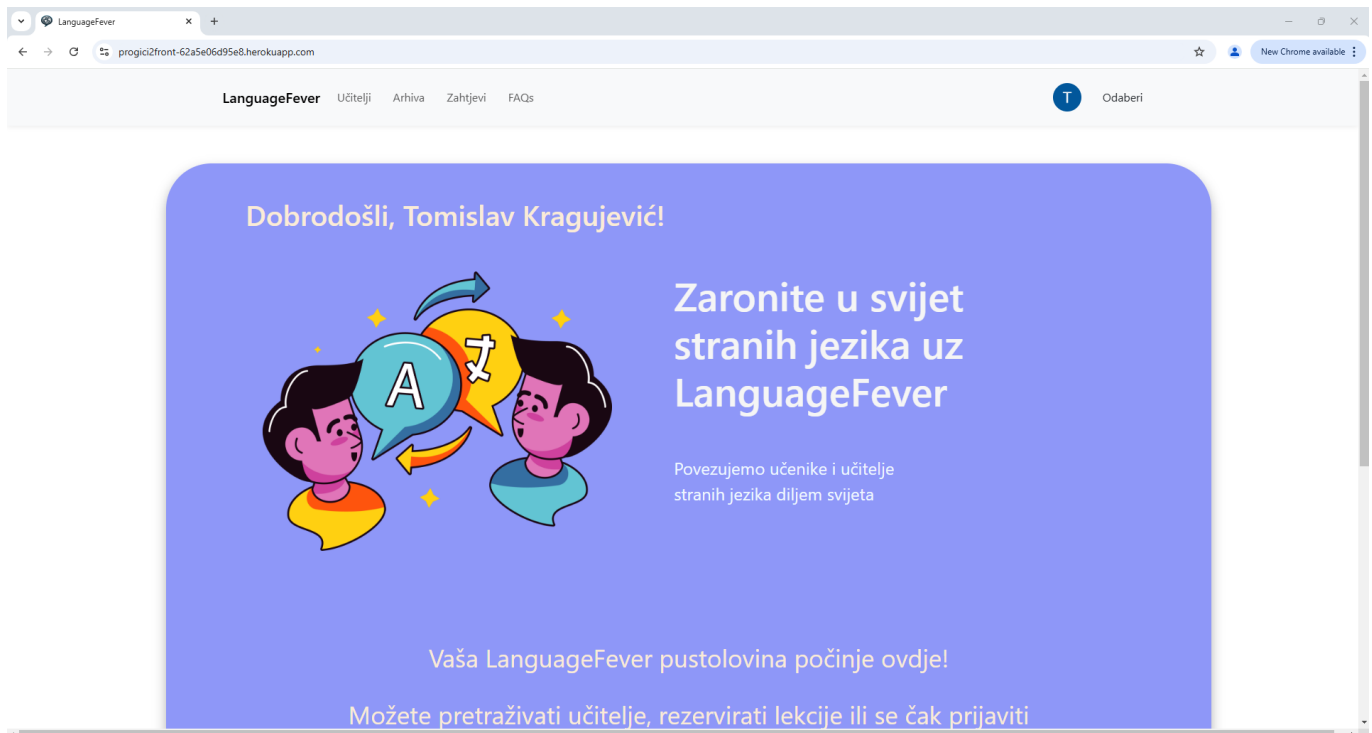
```

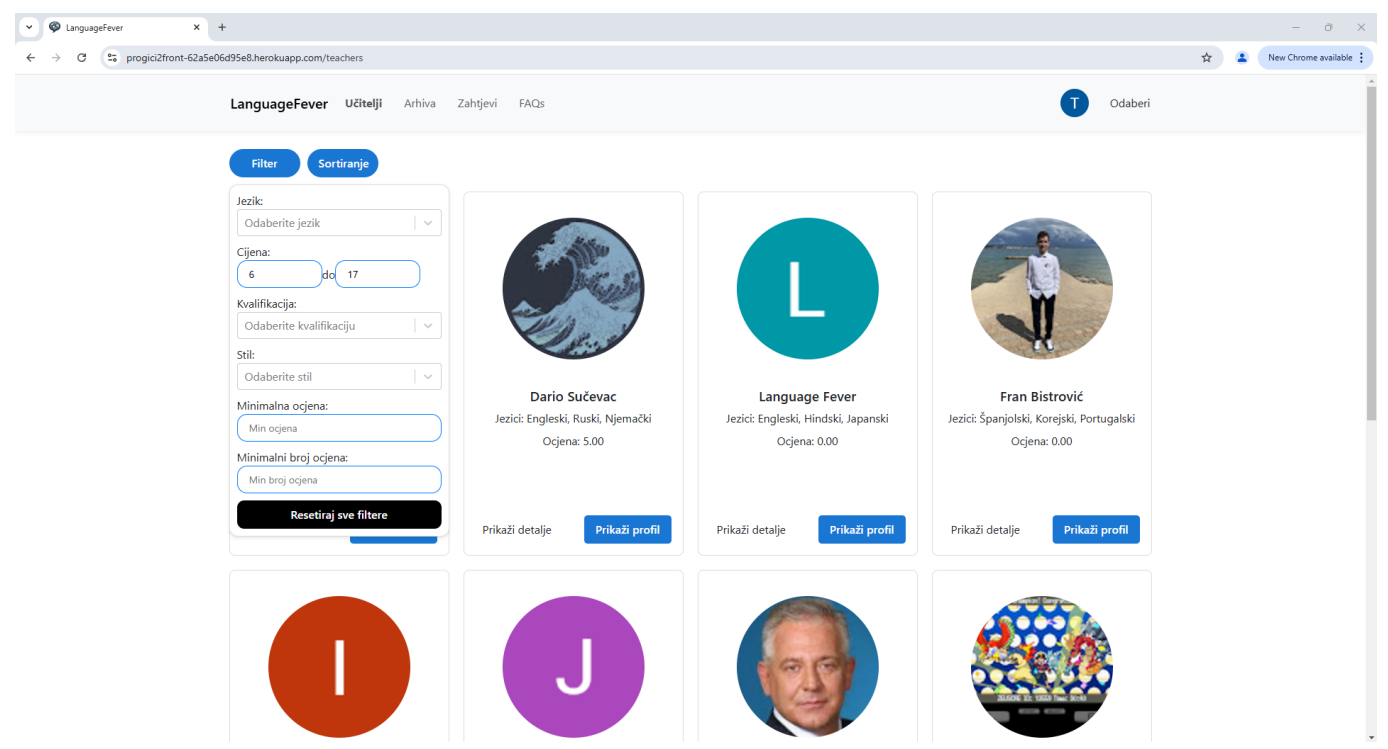
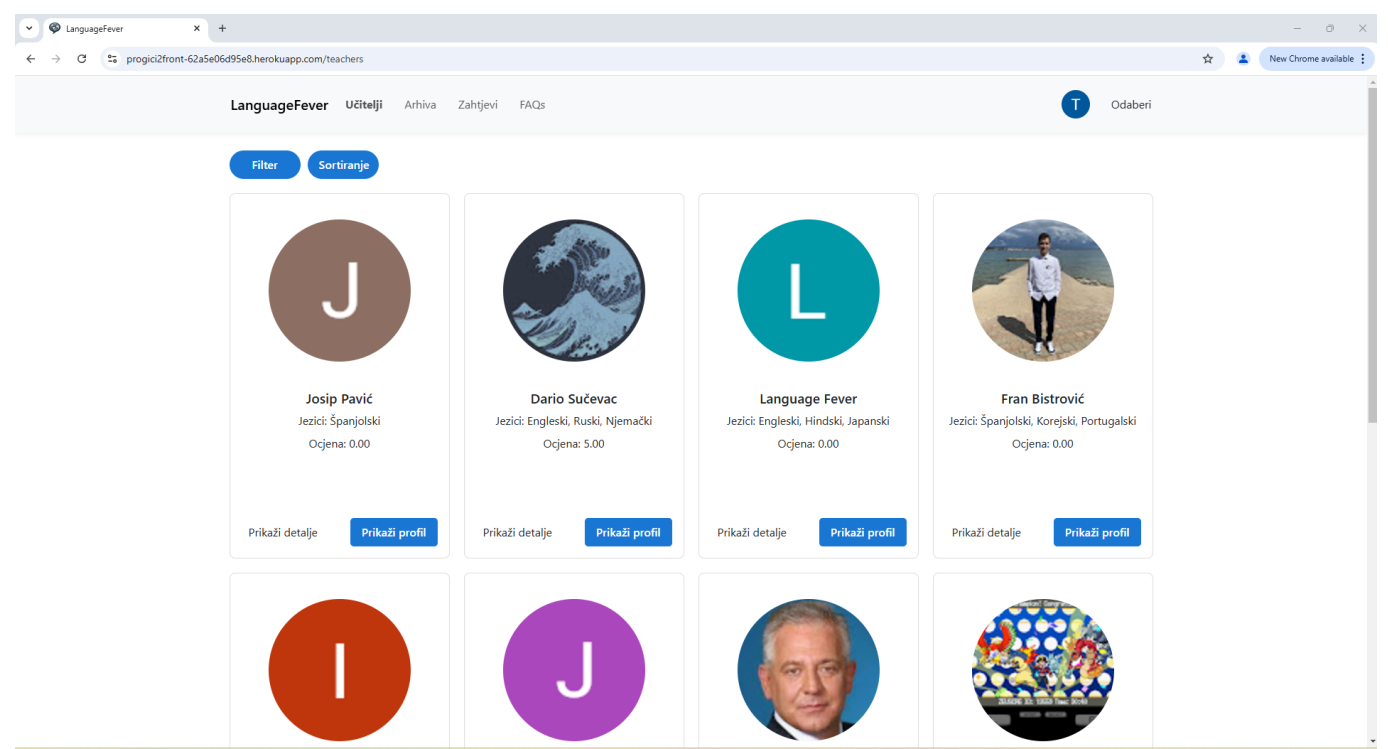
driver.findElement(By.xpath("/html/body/div/div/div[1]/div/div[2]/label[2]/div/input[1]")).sendKeys("6");
    try {
        Thread.sleep(2000); // Wait for 2 seconds
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    // Max price

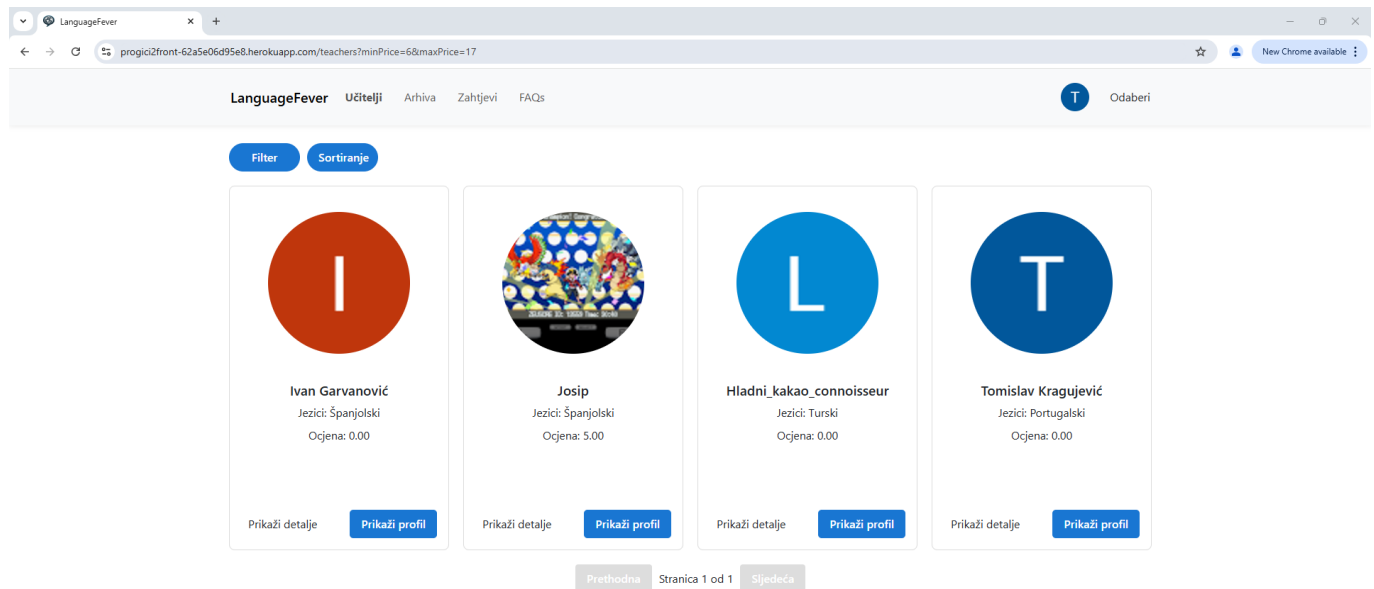
driver.findElement(By.xpath("/html/body/div/div/div[1]/div/div[2]/label[2]/div/input[2]")).sendKeys("17");
    try {
        Thread.sleep(2000); // Wait for 2 seconds
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    // Klikni izvan
driver.findElement(By.xpath("/html/body/div/div")).click();
    try {
        Thread.sleep(2000); // Wait for 2 seconds
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
}

```

## ► PRIKAŽI SLIKE







## Prezentacija rezultata

Za oba tipa ispitivanja (komponenti i sustava) potrebno je:

- Jasno dokumentirati ulaze, korake, očekivane i dobivene rezultate.
- Priložiti slike ekrana, logove ili izvješća generirana alatima (npr. JUnit ili Selenium).
- Detaljno opisati ponašanje sustava, posebno u rubnim uvjetima.
- Navesti broj otkrivenih grešaka!

# Korištene tehnologije i alati

---

Za razvoj i održavanje aplikacije, korišten je **Visual Studio Code (VSC)**, integrirano razvojno okruženje (IDE) koje je široko korišteno među programerima zbog svoje fleksibilnosti i bogatstva ekstenzija. Visual Studio Code omogućava rad s različitim tehnologijama kao što su JavaScript, TypeScript, Java i drugi programski jezici, te pruža alate poput debugiranja, IntelliSense-a za automatsko dovršavanje koda i podrške za verzioniranje putem Gita. VSC je korišten kako za razvoj frontend dijela aplikacije (ReactJS s JavaScript-om) tako i za backend (Spring Boot). Više o Visual Studio Code-u možete saznati i preuzeti na službenoj stranici Visual Studio Code (<https://code.visualstudio.com/>).

Udaljeni repozitorij je dostupan na platformi **GitHub** (<https://github.com/>), gdje je svaki član tima obilježavao svoje promjene na projektu.

## Frontend

Frontend je napravljen putem **ReactJS-a**, JavaScript knjižnice koja omogućava izradu dinamičnih i interaktivnih korisničkih sučelja. ReactJS se temelji na komponentama koje omogućuju ponovnu upotrebu koda i bolju organizaciju aplikacije. Glavna prednost React-a je upotreba virtualnog DOM-a koji omogućava brzo ažuriranje korisničkog sučelja bez ponovnog učitavanja cijele stranice, što rezultira boljim performansama aplikacije. Ovaj framework je idealan za izradu složenih aplikacija koje zahtijevaju visoku interaktivnost i dinamičnost. Više o ReactJS-u možete saznati na stranici ReactJs (<https://reactjs.org/>).

Također su korišteni predlošci za React komponente, **Material UI** (<https://mui.com/>).

## Backend

Za stvaranje backend-a korišten je **Spring Boot**, razvojni okvir za Javu koji olakšava izradu stand-alone aplikacija. Spring Boot automatizira mnoge aspekte konfiguracije i omogućuje brzu implementaciju server-side funkcionalnosti poput RESTful API-ja, upravljanja bazama podataka i drugih ključnih backend operacija. Korištenjem Spring Boot-a, aplikacija može biti brzo postavljena u proizvodnju bez potrebe za složenim konfiguracijama. Više o Spring Boot-u možete pročitati na Spring Boot (<https://spring.io/projects/spring-boot>).

Za testiranje je korišten **Postman** (<https://www.postman.com/>).

## Baza podataka

Za pohranu podataka korištena je **PostgreSQL**, open-source sustav za upravljanje relacijskim bazama podataka. PostgreSQL je poznat po svojoj stabilnosti, visokom stupnju pouzdanosti i podršci za napredne funkcionalnosti kao što su transakcije, ACID svojstva i rad s velikim količinama podataka. Koristi se u aplikacijama koje zahtijevaju visok stupanj integriteta podataka. Za rad s PostgreSQL-om korišten je alat **pgAdmin4**, grafičko sučelje za administraciju baze podataka. Za više informacija o PostgreSQL-u i preuzimanje, posjetite PostgreSQL (<https://www.postgresql.org/>).

## Deploy

Aplikacija je postavljena u produkciju putem **Heroku**, cloud platforme koja omogućava jednostavno hostanje i skaliranje aplikacija. Heroku uklanja potrebu za rukovanjem infrastrukturom i omogućuje brzu implementaciju

aplikacije u oblaku, što omogućava da aplikacija bude odmah dostupna korisnicima. Heroku podržava razne programske jezike, uključujući Java i JavaScript, što ga čini pogodnim za razne vrste aplikacija. Više o Heroku platformi možete saznati na Heroku (<https://www.heroku.com/>).

## Ispitivanje

Uz testiranje pomoću Postmana za provjeru RESTful API-ja, za end-to-end testiranje korisničkog sučelja korišten je i Selenium. Selenium je open-source okvir za automatizirano testiranje web aplikacija koji omogućava simulaciju korisničkih akcija (klikovi, unosi u polja, navigacija po stranicama i sl.) u različitim preglednicima. Primjenom Selenija moguće je pravovremeno otkriti i ispraviti potencijalne pogreške u frontend dijelu aplikacije, čime se osigurava bolja kvaliteta proizvoda i zadovoljstvo korisnika. Testovi pisani u Seleniumu mogu se lako integrirati u postojeće CI/CD procese, što omogućava kontinuirano testiranje i brzu isporuku aplikacijskih rješenja. Više informacija o Seleniumu možete pronaći na Seleniumu (<https://www.selenium.dev/>).

## Dokumentacija

Za izradu i održavanje dokumentacije korišten je **GitHub Wiki**, alat unutar GitHub-a koji omogućuje jednostavno kreiranje i uređivanje stranica s dokumentacijom za projekte. GitHub Wiki pruža pregledan način za pohranu tehničkih informacija, vodiča i drugih važnih podataka koji olakšavaju razvoj i održavanje aplikacije. Svi ključni detalji o arhitekturi sustava, uputama za implementaciju i drugim tehničkim informacijama pohranjeni su u GitHub Wiki-u, što omogućuje lako praćenje i zajednički rad na dokumentaciji. Više o GitHub Wiki-u možete saznati na [GitHub Wiki](#).

## Komunikacija

Komunikacija između članova tima ostvarena je putem WhatsApp-a (<https://www.whatsapp.com/>) te Discord-a (<https://discord.com/>), a s asistentima putem Microsoft Teams-a (<https://www.microsoft.com/hr-hr/microsoft-teams/group-chat-software>).

---

## Lokalno pokretanje aplikacije

### 1. Preuvjeti

Prije početka, provjerite imate li instalirane sljedeće alate te su navedene verzije koje su provjerene da rade:

- **Node.js:** verzija 23
- **Java:** verzija 17
- **Git:** verzija 2.39.5
- **PostgreSQL:** verzija 14

---

### 2. Preuzimanje izvornog koda

Za preuzimanje izvornog koda projekta, slijedite ove korake:

1. Klonirajte repozitorij:

```
git clone https://github.com/Progici/LanguageFever.git
```

2. Postavite **API\_URL** u datoteku **ApiConfig**:

```
API_URL: "http://localhost:8080"
```

3. Uđite u direktorij projekta:

```
cd LanguageFever
```

---

### 3. Postavljanje baze podataka

1. Povežite se s PostgreSQL serverom i stvorite novu bazu podataka:

```
psql -h localhost -p 5433 -U postgres -d postgres
```

2. Unesite lozinku (po defaultu: 0000).

3. Stvorite novu bazu podataka:

```
CREATE DATABASE languagefever;
```

#### 4. Ažurirajte **application.properties**:

```
spring.datasource.url=jdbc:postgresql://localhost:5433/languagefever  
spring.datasource.username=postgres  
spring.datasource.password=0000
```

#### 5. Dodajte Google i GitHub klijentske tajne u **application.properties**:

```
google.client.secret=YOUR_GOOGLE_SECRET  
github.client.secret=YOUR_GITHUB_SECRET
```

---

## 4. Instalacija ovisnosti

### Backend:

#### 1. Instalirajte potrebne ovisnosti:

```
./mvnw clean install
```

#### 2. Pokrenite aplikaciju:

```
./mvnw spring-boot:run
```

---

### Frontend:

#### 1. Instalirajte potrebne ovisnosti:

```
npm install
```

#### 2. Pokrenite razvojni server:

```
npm run dev
```

---

## Deploy aplikacije na Heroku

Projekt je pripremljen za deploy na Heroku s dvije aplikacije: **poslužiteljski dio (backend)** i **klijentski dio (frontend)**.

## Postavljanje poslužiteljske aplikacije (Backend)

1. Postavite **Config Vars** na Heroku za backend aplikaciju:

- **Key:** `GITHUB_SECRET`  
**Value:** `YOUR_GITHUB_CLIENT_SECRET`
- **Key:** `GCLIENT_SECRET`  
**Value:** `YOUR_GOOGLE_CLIENT_SECRET`
- **Key:** `FRONTEND_URL`  
**Value:** `YOUR_HEROKU_FRONTEND_URL`
- **Key:** `Procfile`  
**Value:** `Procfile.backend`

2. Dodajte Heroku buildpack za Java.

3. Dodajte PostgreSQL add-on svom Heroku projektu.

4. Spojite se na GitHub, odaberite odgovarajući repozitorij i granu **master**, a zatim kliknite **Deploy**.

---

## Postavljanje klijentske aplikacije (Frontend)

1. Postavite **Config Vars** na Heroku:

- **Key:** `Procfile`  
**Value:** `Procfile.frontend`
- **Key:** `VITE_BACKEND_URL`  
**Value:** `YOUR_HEROKU_BACKEND_URL`

2. Dodajte Heroku buildpack za Node.js.

3. Spojite se na GitHub, odaberite odgovarajući repozitorij i granu **master**, a zatim kliknite **Deploy**.

---